

EC ENG 232E - Summer 25 - Project 2

July 25, 2025

Team members:

Kaylee Lo (UID: _____)

Joanna Tang (UID: _____)

Rachel Zhang (UID: _____)

1 1. Facebook network

In this project, we will be using the dataset given below: <http://snap.stanford.edu/data/egonets-Facebook.html> The Facebook network can be created from the edgelist file (facebook combined.txt)

1.0.1 1.1 Structural properties of the Facebook network

Having created the Facebook network, we will study some of the structural properties of the network. To be specific, we will study • Connectivity • Degree distribution

1.0.2 1: A first look at the network:

Q1.1: Report the number of nodes and number of edges of the Facebook network.

Q1.2: Is the Facebook network connected? If not, find the GCC of the network and report the size of the GCC.

Q2: Find the diameter of the network. If the network is not connected, then find the diameter of the GCC.

Q3: Plot the degree distribution of the facebook network and report the average degree.

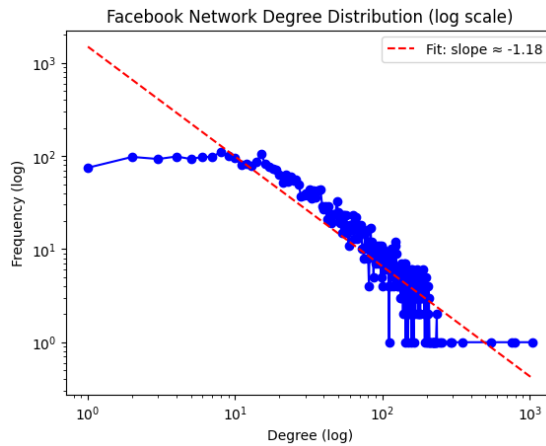
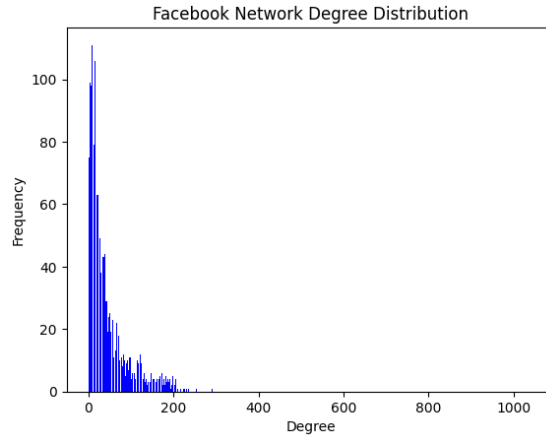
Q4: Plot the degree distribution of Question 3 in a log-log scale. Try to fit a line to the plot and estimate the slope of the line.

1.1 Number of nodes: 4039 Number of edges: 88234

1.2 Is connected: True , size of connected components: 4039

2. Diameter: 8

3. Average degree: 43.69101262688784



Slope of the line: -1.18

- 1.1) Number of nodes: 4039, Number of edges: 88234
- 1.2) Is connected: True , size of connected components: 4039
- 2) Diameter: 8
- 3) Average degree: 43.69
- 4) Slope of the best-fit line: -1.18

1.1 2. Personalized network

A personalized network of an user v_i is defined as the subgraph induced by v_i and it's neighbors. In this part, we will study some of the structural properties of the personalized network of the user whose graph node ID is 1 (node ID in edgelist is 0). From this point onwards, whenever we are referring to a node ID we mean the graph node ID which is $1 + \text{node ID in edgelist}$.

1.1.1 Q5. Create a personalized network of the user whose ID is 1.

How many nodes and edges does this personalized network have?

Number of nodes: 348
Number of edges: 2866

Answer: Personalized network ID 1 has 348 nodes and 2866 edges.

1.1.2 Q6. What is the diameter of the personalized network?

Please state a trivial upper and lower bound for the diameter of the personalized network.

Diameter: 2
Trivial Lower Bound Diameter: 1
Trivial Upper Bound Diameter: 2

Answer: Personalized network ID 1 has a diameter of 2, a trivial lower bound of 1, and a trivial upper bound of 2.

1.1.3 Q7. In the context of the personalized network, what is the meaning of the diameter of the personalized network to be equal to the upper bound you derived in Question 6.

What is the meaning of the diameter of the personalized network to be equal to the lower bound you derived in Question 6 (assuming there are more than 3 nodes in the personalized network)?

Answer:

If the diameter of the network, ie. the longest shortest path between any two nodes in the network, is equal to the UPPER BOUND, then that means that the nodes are not neighbors of each other, and are connected by the core node. (ex: $\text{path}(n1, n2)$ would be $n1 \rightarrow \text{core node} \rightarrow n2$)

If the diameter of is equal to the LOWER BOUND, then the farthest two nodes are equal to the shortest-path between any pair of nodes. The trivial lower bound is the smallest length of two nodes directly connected to each other. This means that all nodes are neighbors of each other.

1.2 3. Core node's personalized network

1.2.1 Q8. How many core nodes are there in the Facebook network.

What is the average degree of the core nodes?

There are 41 core nodes in the Facebook network
Average degree of the core nodes: 277.4390243902439

Answer: There are 41 core nodes in the network, with an average degree of 277.44.

1.3 3.1. Community structure of core node's personalized network

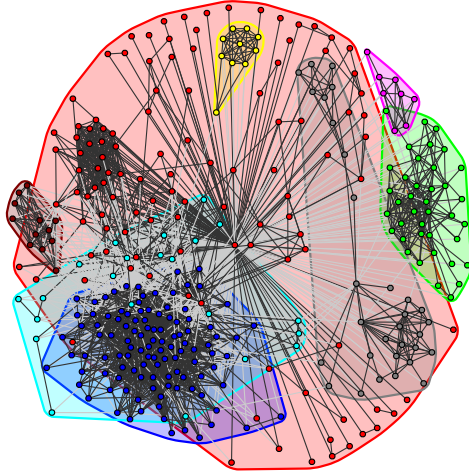
In this part, we study the community structure of the core node's personalized network. To be specific, we will study the community structure of the personalized network of the following core nodes:

• Node ID 1 • Node ID 108 • Node ID 349 • Node ID 484 • Node ID 1087

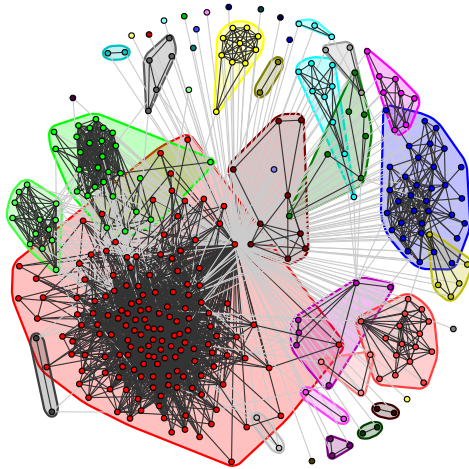
1.3.1 Q9: For each of the above core node's personalized network, find the community structure using Fast-Greedy, Edge-Betweenness, and Infomap community detection algorithms.

Compare the modularity scores of the algorithms. For visualization purpose, display the community structure of the core node's personalized networks using colors. Nodes belonging to the same community should have the same color and nodes belonging to different communities should have different color. In this question, you should have 15 plots in total.

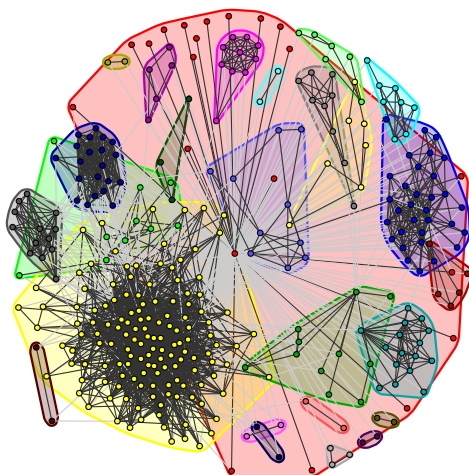
Node 1: Fast-Greedy



Node 1: Edge-betweenness



Node 1: Infomap



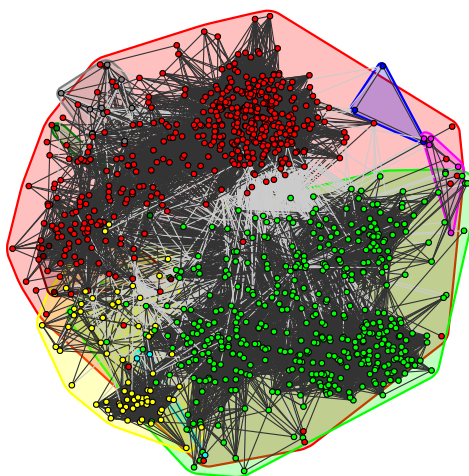
Node 1 Modularity scores:

Fast-greedy: 0.41310137283423476

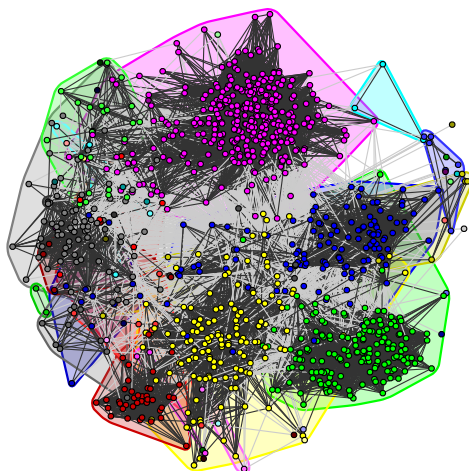
Edge-betweenness: 0.3533021725463346

Infomap: 0.38911847105097724

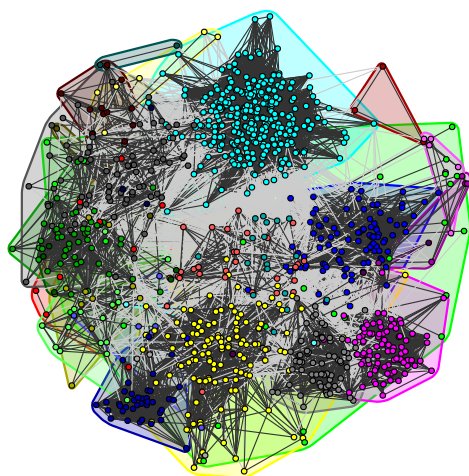
Node 108: Fast-Greedy



Node 108: Edge-betweenness



Node 108: Infomap



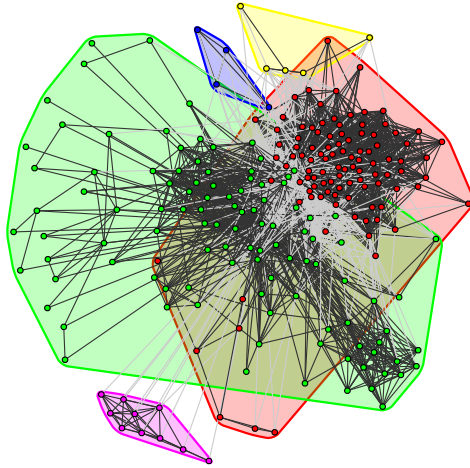
Node 108 Modularity scores:

Fast-greedy: 0.435929376026475

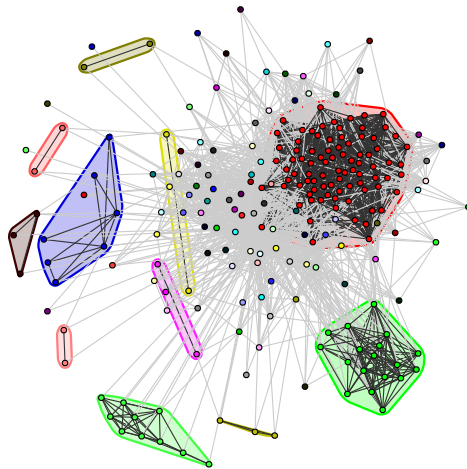
Edge-betweenness: 0.5067549165389017

Infomap: 0.508249171320581

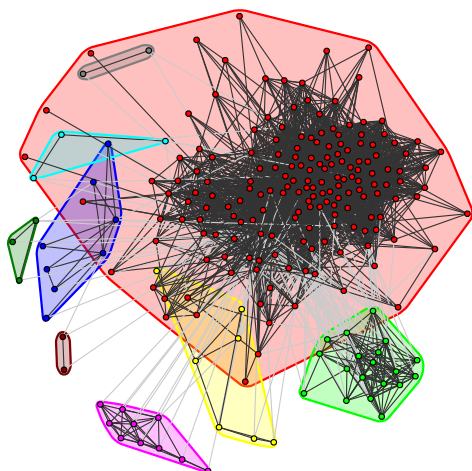
Node 349: Fast-Greedy



Node 349: Edge-betweenness

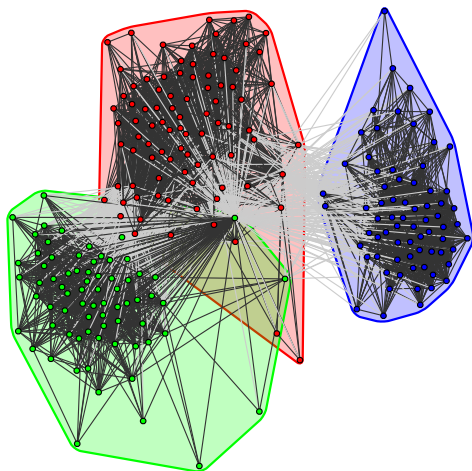


Node 349: Infomap

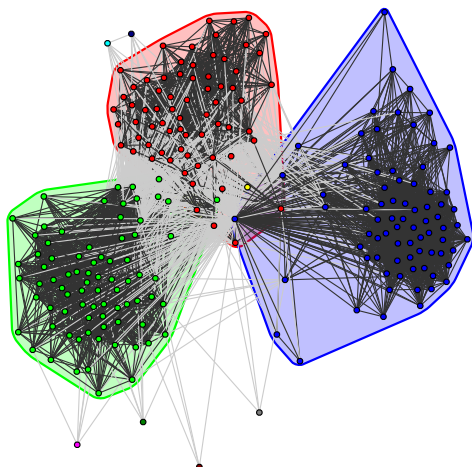


Node 349 Modularity scores:
 Fast-greedy: 0.2503460796905125
 Edge-betweenness: 0.13352802137007763
 Infomap: 0.09546419609135802

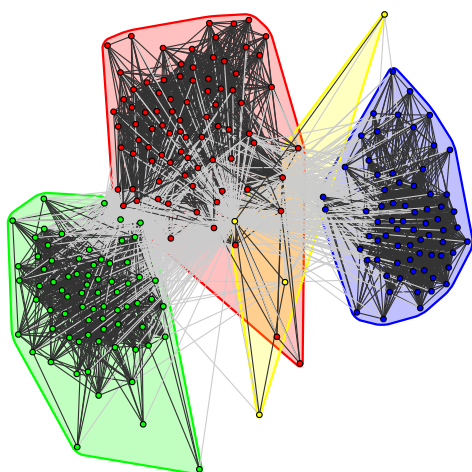
Node 484: Fast-Greedy



Node 484: Edge-betweenness



Node 484: Infomap



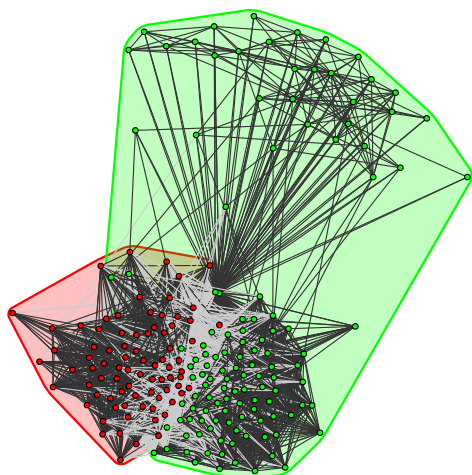
Node 484 Modularity scores:

Fast-greedy: 0.5070016421965142

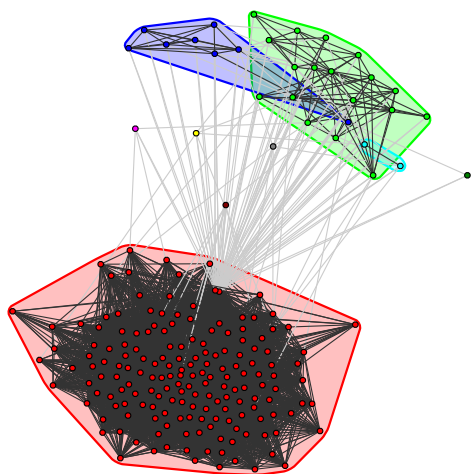
Edge-betweenness: 0.48909518024480314

Infomap: 0.515278752174842

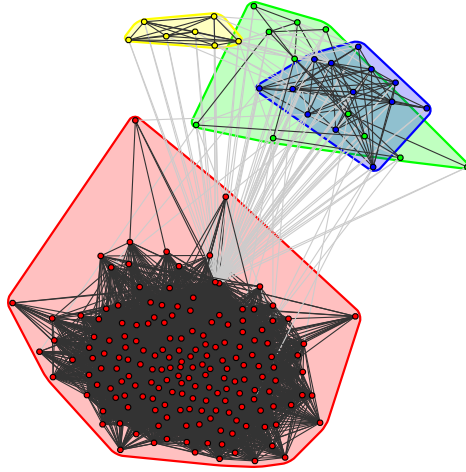
Node 1087: Fast-Greedy



Node 1087: Edge-betweenness



Node 1087: Infomap



Node 1087 Modularity scores:
Fast-greedy: 0.14553149956549316
Edge-betweenness: 0.02762377238846389
Infomap: 0.026906617223335445

1.3.2 Q9 continued: Compare the modularity scores of the algorithms.

Summary of modularity scores results:

Node ID	Fast-Greedy	Edge-Betweenness	Infomap
1	0.413	0.353	0.389
108	0.436	0.507	0.508
349	0.250	0.134	0.095
484	0.507	0.489	0.515
1087	0.146	0.028	0.027

Answer: Generally, the community structure found using the fast-greedy algorithm has the highest modularity. Edge-betweenness and infomap result in similar modularity scores. However, we can see that for the personalized networks for nodes 108 and 484, infomap performs better than the other two algorithms. This suggests that infomap may be the preferred algorithm for larger networks.

1.4 3.2 Community structure with the core node removed

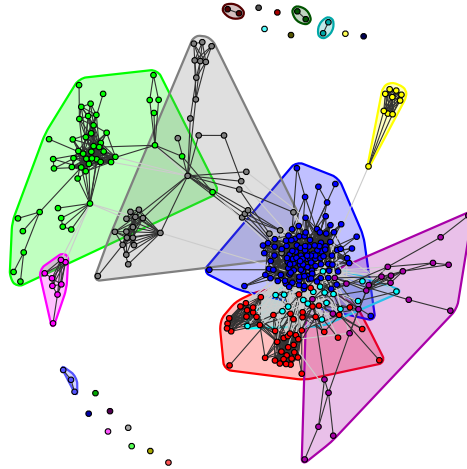
In this part, we will explore the effect on the community structure of a core node's personalized network when the core node itself is removed from the personalized network.

1.4.1 Q10: For each of the core node's personalized network (use same core nodes as Question 9), remove the core node from the personalized network and find the community structure of the modified personalized network.

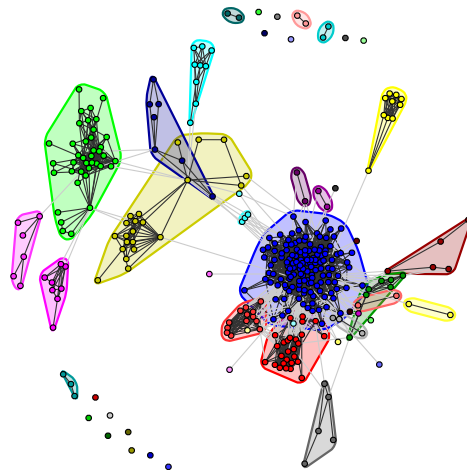
Use the same community detection algorithm as Question 9. For visualization purpose, display the community structure of the modified personalized network using colors. In this question, you should have 15 plots in total.

Community structures with Node 1 removed

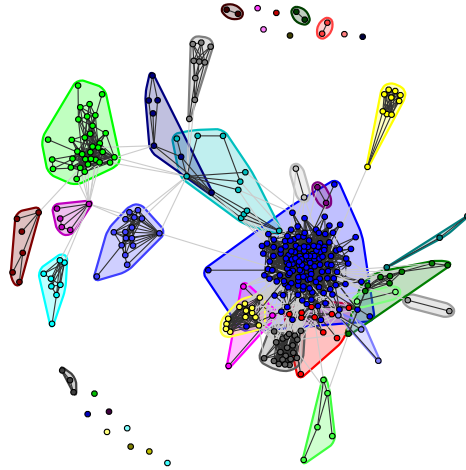
Node 1: Fast-Greedy



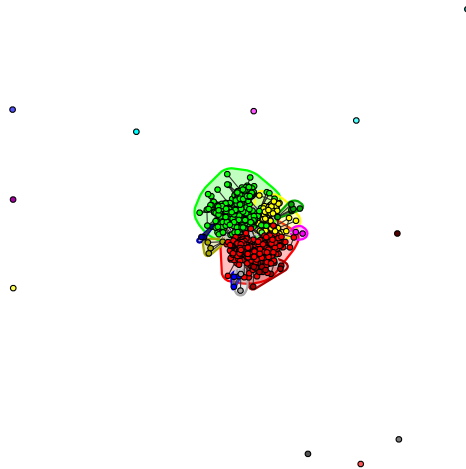
Node 1: Edge-betweenness



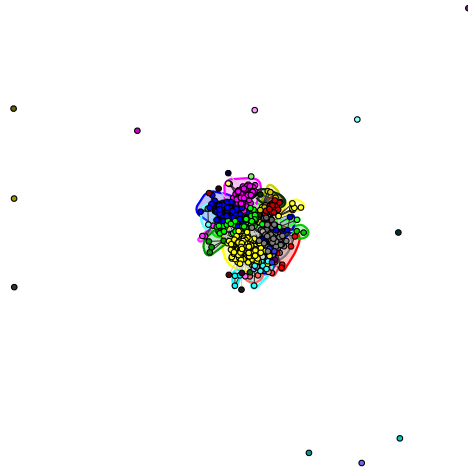
Node 1: Infomap



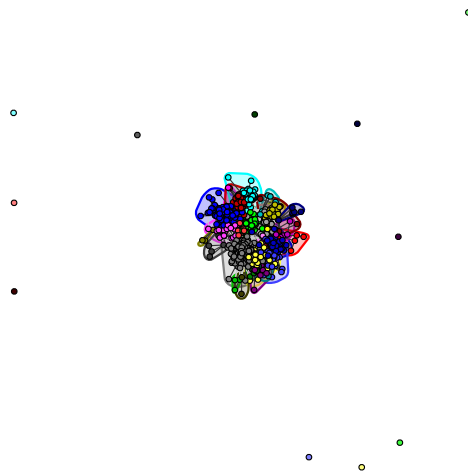
Node 1 removed Modularity scores:
 Fast-greedy: 0.44185326886839066
 Edge-betweenness: 0.4161461420398303
 Infomap: 0.4180076594538908
 Community structures with Node 108 removed
 Node 108: Fast-Greedy



Node 108: Edge-betweenness

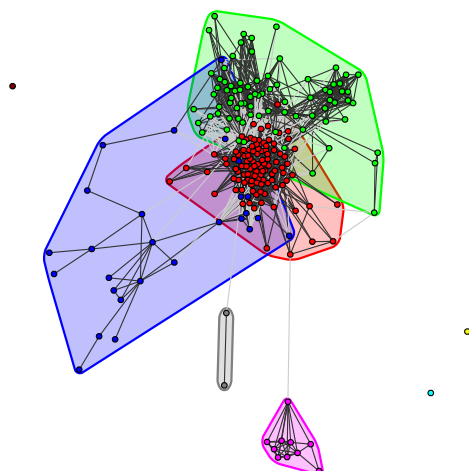


Node 108: Infomap

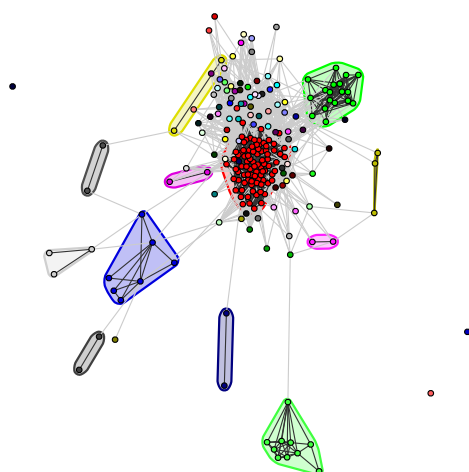


Node 108 removed Modularity scores:
 Fast-greedy: 0.45812709371997556
 Edge-betweenness: 0.5213215763822159
 Infomap: 0.5179958277578826

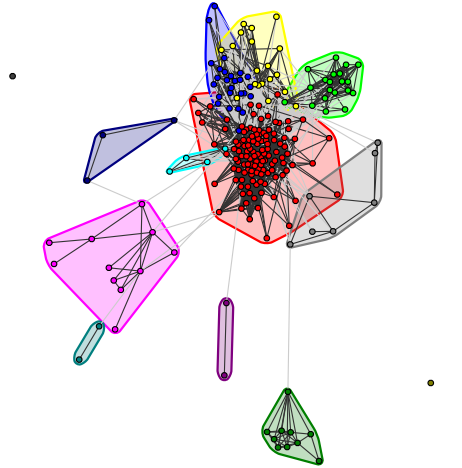
Community structures with Node 349 removed
 Node 349: Fast-Greedy



Node 349: Edge-betweenness

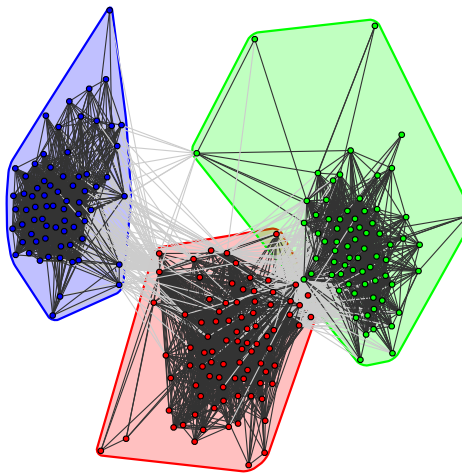


Node 349: Infomap

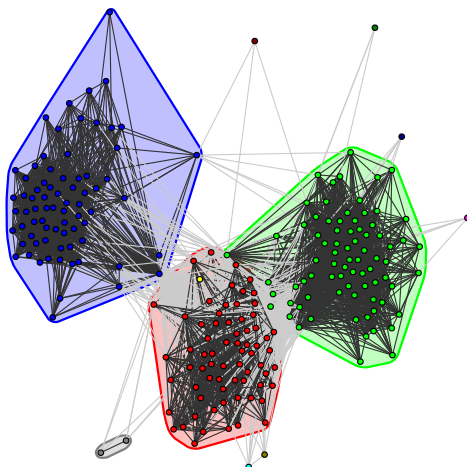


Node 349 removed Modularity scores:
 Fast-greedy: 0.24569179594267446
 Edge-betweenness: 0.15056634018755946
 Infomap: 0.23377319873016644

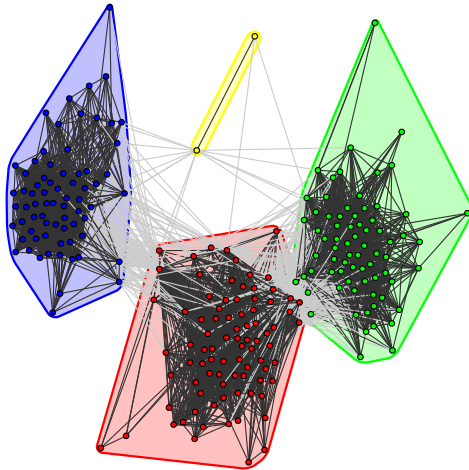
Community structures with Node 484 removed
 Node 484: Fast-Greedy



Node 484: Edge-betweenness

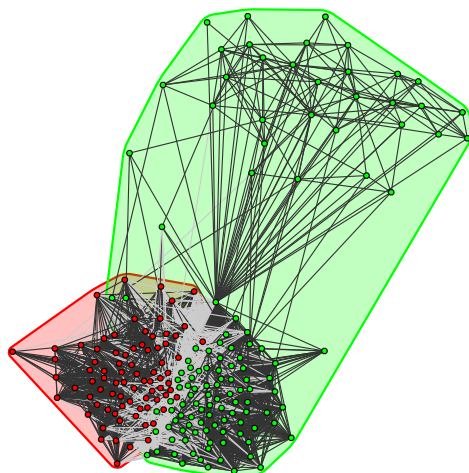


Node 484: Infomap

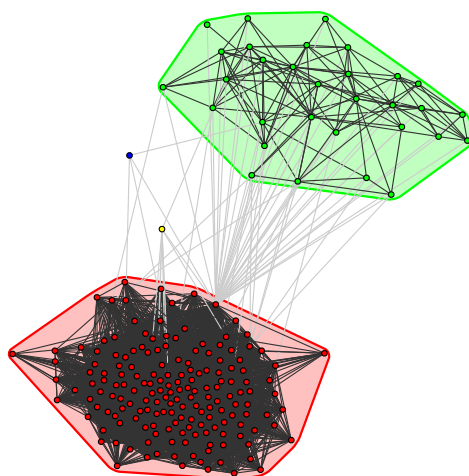


Node 484 removed Modularity scores:
 Fast-greedy: 0.5393569443742408
 Edge-betweenness: 0.5154412771235044
 Infomap: 0.5434436792795223

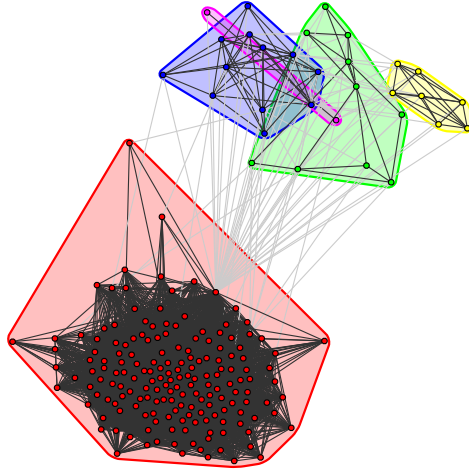
Community structures with Node 1087 removed
 Node 1087: Fast-Greedy



Node 1087: Edge-betweenness



Node 1087: Infomap



Node 1087 removed Modularity scores:
Fast-greedy: 0.1481956319534985
Edge-betweenness: 0.03249529804991398
Infomap: 0.02737159448711458

1.4.2 Q10 continued: Compare the modularity score of the community structure of the modified personalized network with the modularity score of the community structure of the personalized network of Question 9.

Summary of results:

Node ID	Fast-Greedy	Edge-Betweenness	Infomap
1	0.413	0.353	0.389
1 (without)	0.442	0.416	0.418
108	0.436	0.507	0.508
108 (without)	0.458	0.521	0.518
349	0.250	0.134	0.095
349 (without)	0.246	0.151	0.234
484	0.507	0.489	0.515
484 (without)	0.539	0.515	0.543
1087	0.146	0.028	0.027
1087 (without)	0.148	0.032	0.027

Answer: For fast-greedy, the modularity score of the personalized network is similar, but usually higher in the network where the core node is removed (with the exception of node 349). For edge-betweenness, the modularity score is higher when the core node is removed from the personalized network. For infomap, the modularity score is also higher when the core node is removed.

1.5 3.3. Characteristic of nodes in the personalized network

In this part, we will explore characteristics of nodes in the personalized network using two measures. These two measures are stated and defined below:

- Embeddedness of a node is defined as the number of mutual friends a node shares with the core node.
- Dispersion of a node is defined as the sum of distances between every pair of the mutual friends the node shares with the core node. The distances should be calculated in a modified graph where the node (whose dispersion is being computed) and the core node are removed.

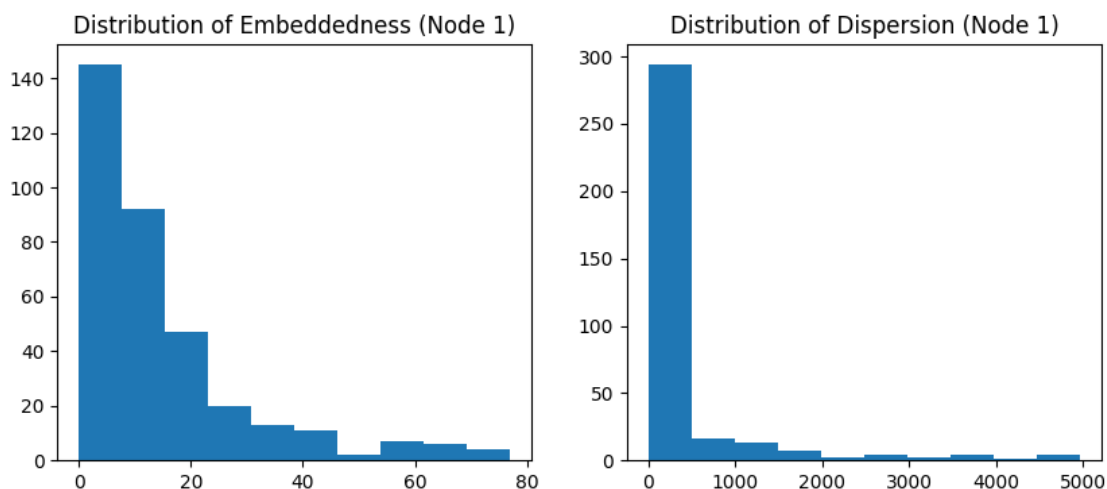
1.5.1 Q11: Write an expression relating the Embeddedness between the core node and a non-core node to the degree of the non-core node in the personalized network of the core node.

Answer: $\text{Embeddedness}(\text{core_node}, \text{node}) = \text{deg}(\text{node}) - 1$. Since the core node is “friends” with all the nodes in the network, the number of mutual friends a node shares with the core node would be the $\text{degree}(\text{node}) - 1$, since the core node would not count as a mutual friend.

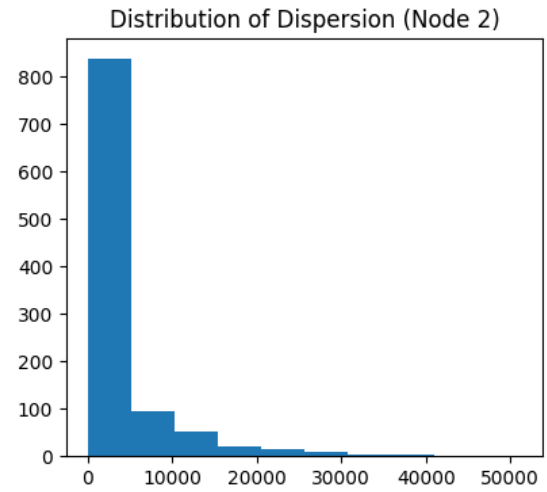
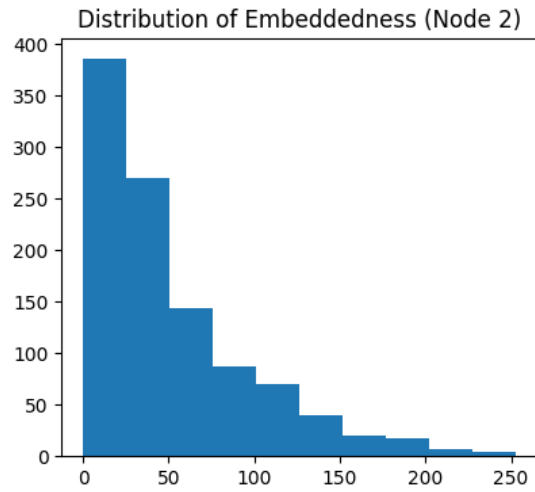
1.5.2 Q12: For each of the core node’s personalized network (use the same core nodes as Question 9), plot the distribution histogram of embeddedness and dispersion.

In this question, you will have 10 plots.

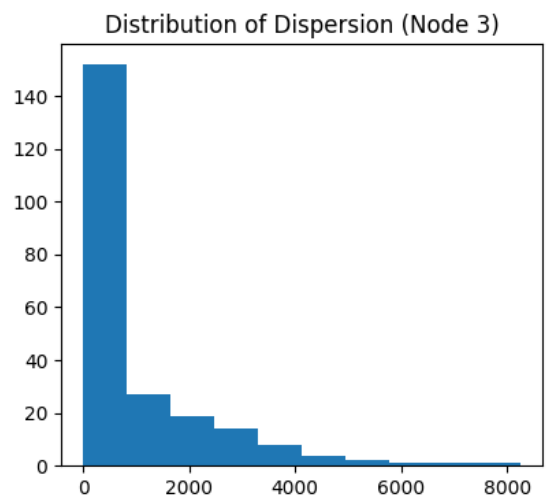
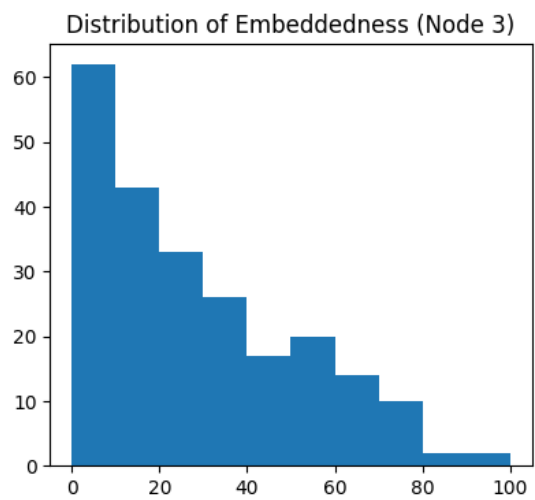
Node 1 Distributions:



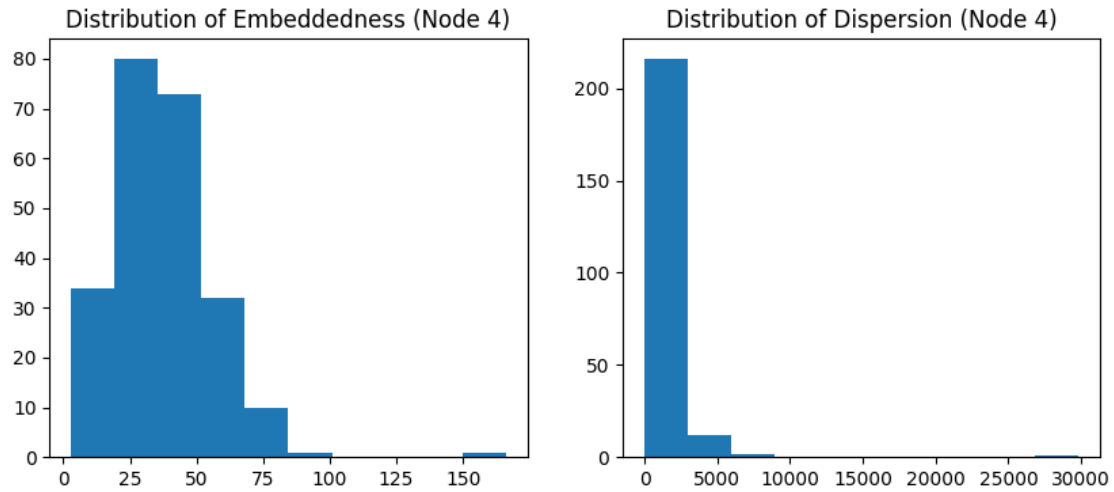
Node 108 Distributions:



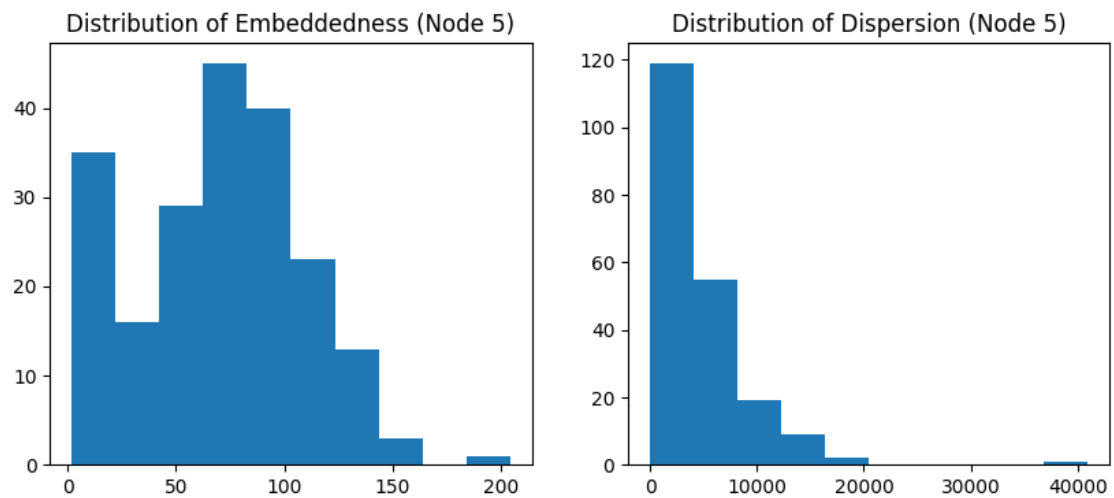
Node 349 Distributions:



Node 484 Distributions:



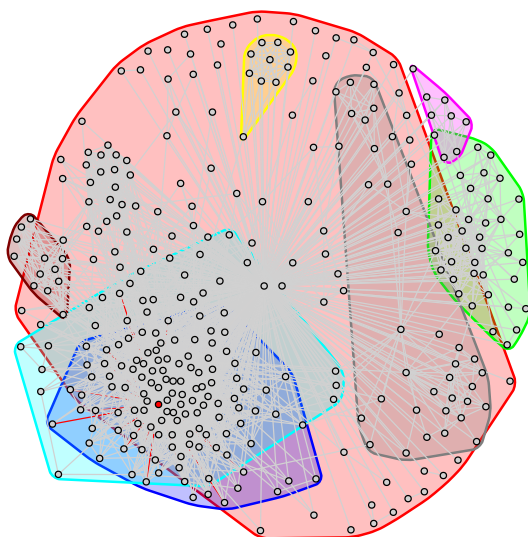
Node 1087 Distributions:



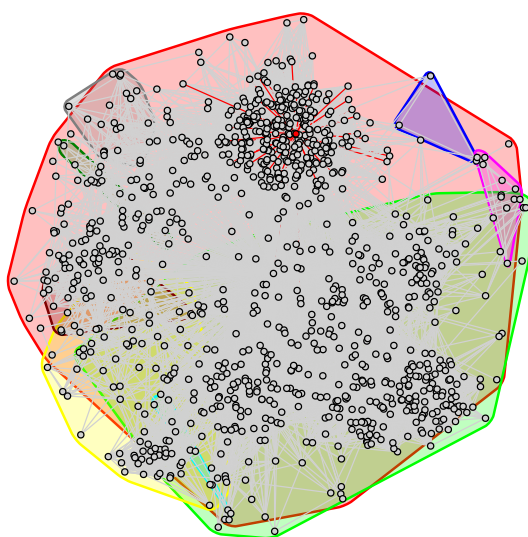
1.5.3 Q13: For each of the core node's personalized network, plot the community structure of the personalized network using colors and highlight the node with maximum dispersion.

Also, highlight the edges incident to this node. To detect the community structure, use Fast-Greedy algorithm. In this question, you will have 5 plots

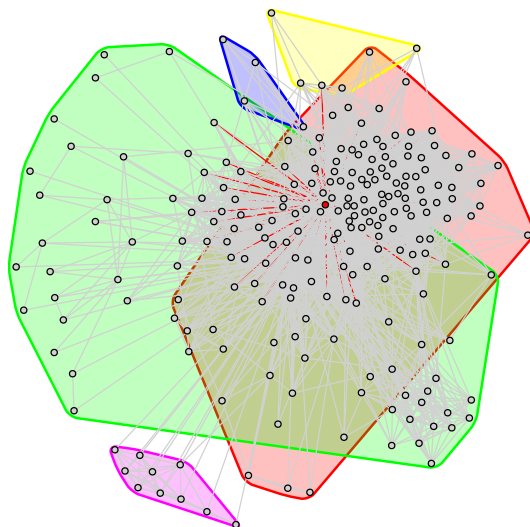
Community structure of personalized network (Node 1) with max dispersion node



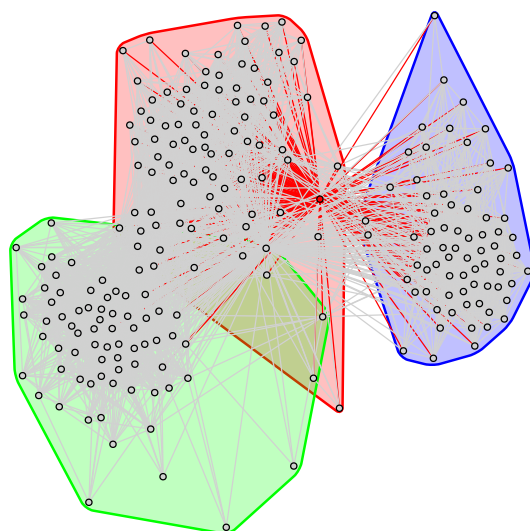
Community structure of personalized network (Node 108) with max dispersion node



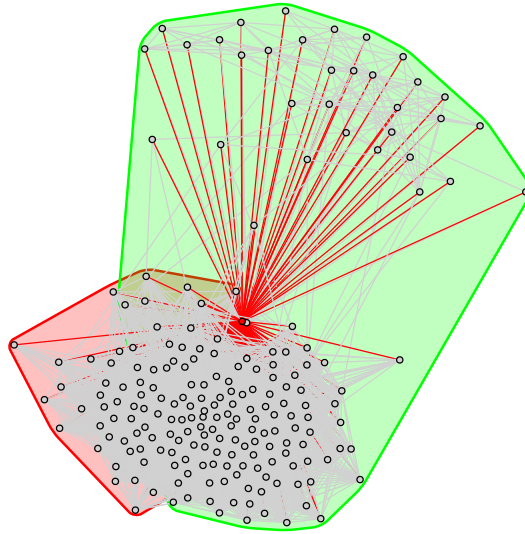
Community structure of personalized network (Node 349) with max dispersion node



Community structure of personalized network (Node 484) with max dispersion node



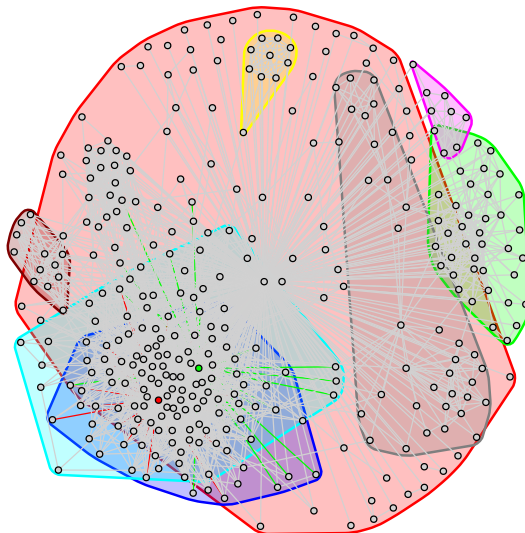
Community structure of personalized network (Node 1087) with max dispersion node



1.5.4 Q14. Repeat Question 13, but now highlight the node with maximum embeddedness and the node with maximum dispersion / embeddedness.

Also, highlight the edges incident to these nodes. Report the id of those nodes.

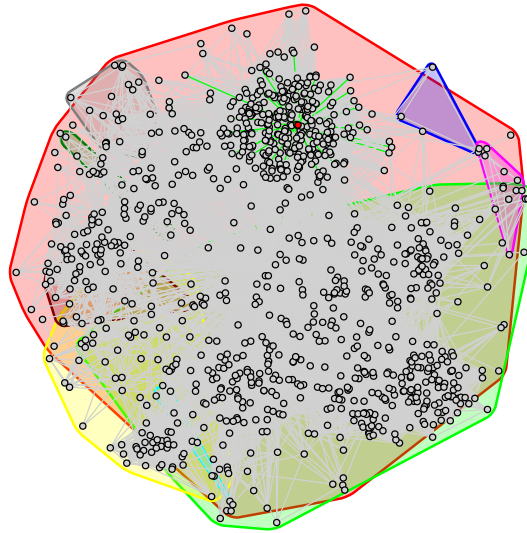
Community structure network (Node 1) with max embeddedness node, max dispersion/embeddedness



Node with maximum embeddedness: 57

Node with maximum dispersion/embeddedness: 26

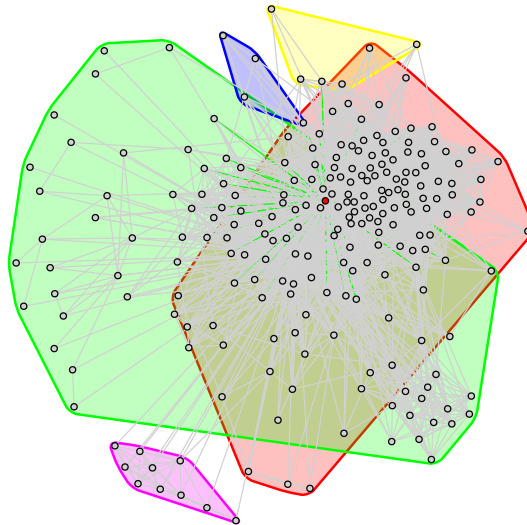
Community structure network (Node 108) with max embeddedness node, max dispersion/embeddedness



Node with maximum embeddedness: 1889

Node with maximum dispersion/embeddedness: 1889

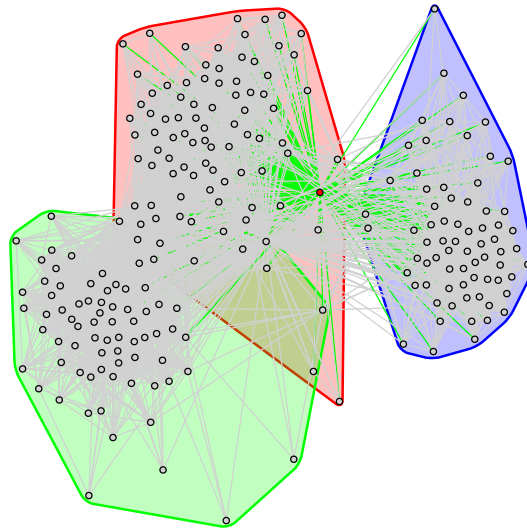
Community structure network (Node 349) with max embeddedness node, max dispersion/embeddedness



Node with maximum embeddedness: 377

Node with maximum dispersion/embeddedness: 377

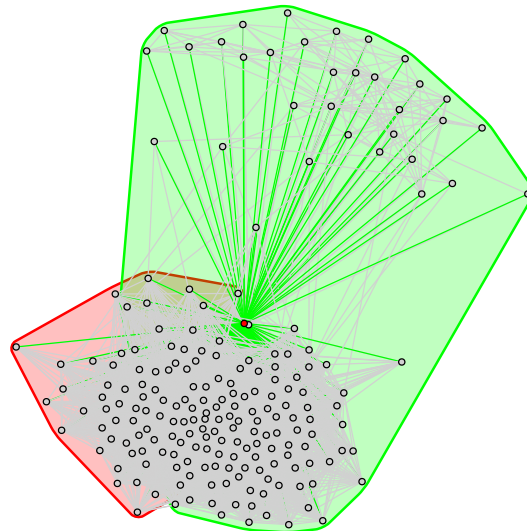
Community structure network (Node 484) with max embeddedness node, max dispersion/embeddedness



Node with maximum embeddedness: 108

Node with maximum dispersion/embeddedness: 108

Community structure network (Node 1087) with max embeddedness node, max dispersion/embeddedness



Node with maximum embeddedness: 108

Node with maximum dispersion/embeddedness: 108

1.5.5 Q15: Use the plots from Question 13 and 14 to explain the characteristics of a node revealed by each of this measure.

Answer: For each network, the node with maximum embeddedness is highly connected with the other nodes in the network - this is evident in the plots where we highlight the edges incident to the node. Aside from the core node, the node with maximum embeddedness would have the highest degree.

For the nodes with maximum dispersion, meaning that the node's "friends" are not well connected with each other, we can see in the plots that these nodes are neighbors with various nodes spanning different communities.

For the nodes with maximum dispersion / embeddedness, we can see in the plots that they are often the same node as maximum dispersion. This suggests that a node with high dispersion generally has low embeddedness - ie. a node that has friends that are not well connected with each other would also share few friends with the core node.

1.6 4. Friend recommendation in personalized networks

The network that we will be using for this part is the personalized network of node with ID 415.

Graph with 160 nodes and 1857 edges

1.7 4.3. Creating the list of users

Having defined the friend recommendation procedure, we can now apply it to the personalized network of node ID 415. Before we apply the algorithm, we need to create the list of users who we want to recommend new friends to. We create this list by picking all nodes with degree 24. We will denote this list as N_r .

1.7.1 Q16: What is $|N_r|$, i.e. the length of the list N_r ?

$N_r = [578, 600, 615, 618, 627, 643, 658, 659, 661, 662, 496]$

Length of $|N_r|$: 11

1.8 4.4. Average accuracy of friend recommendation algorithm

1.8.1 Q17. Which friend recommendation algorithm is the best?

Common Neighbors Measure Accuracy: 0.13294904040233518

Jaccard Measure Accuracy: 0.13210696552697007

Adamic Adar Measure Accuracy: 0.13334429731933123

Answer: Based on the average accuracy values, the Adamic Adar Measure performs the best as a friend recommendation algorithm. Although all of the accuracies are similar, the Adamic Adar Measure accuracy is slightly higher than those of the Common Neighbors Measure and Jaccard Measure.

2 2. Google+ network

Setup the network by reading from downloaded files.

2.0.1 Q18: How many personal networks are there?

There are 132 nodes and there are 57 personal networks.

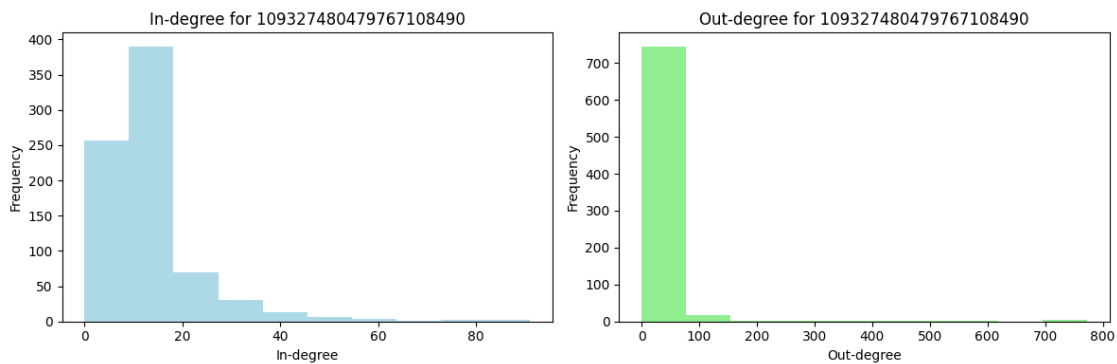
Answer: There are 57 personal networks.

2.0.2 Q19: For the 3 personal networks (node ID given below), plot the in-degree and out-degree distribution of these personal networks.

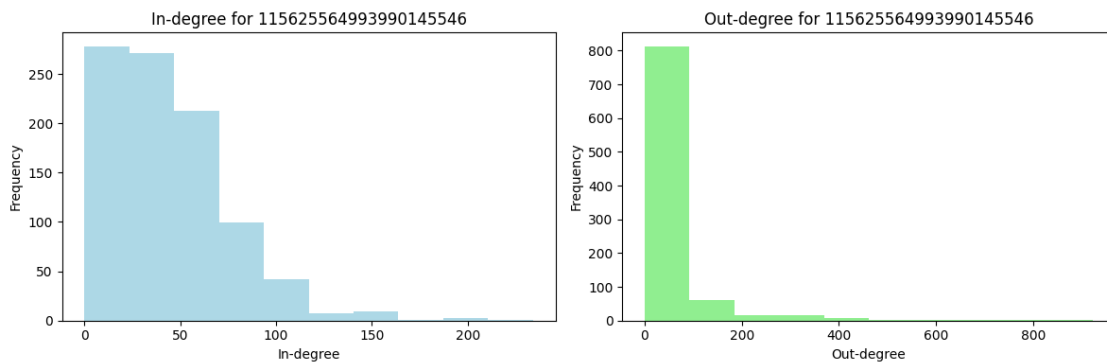
Do the personal networks have a similar in and out degree distribution? In this question, you should have 6 plots.

- 109327480479767108490
- 115625564993990145546
- 101373961279443806744

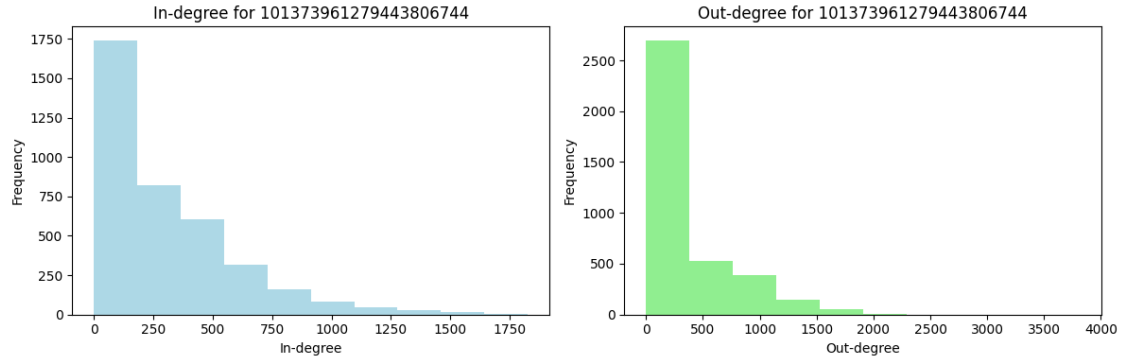
Node index: 69



Node index: 115



Node index: 17



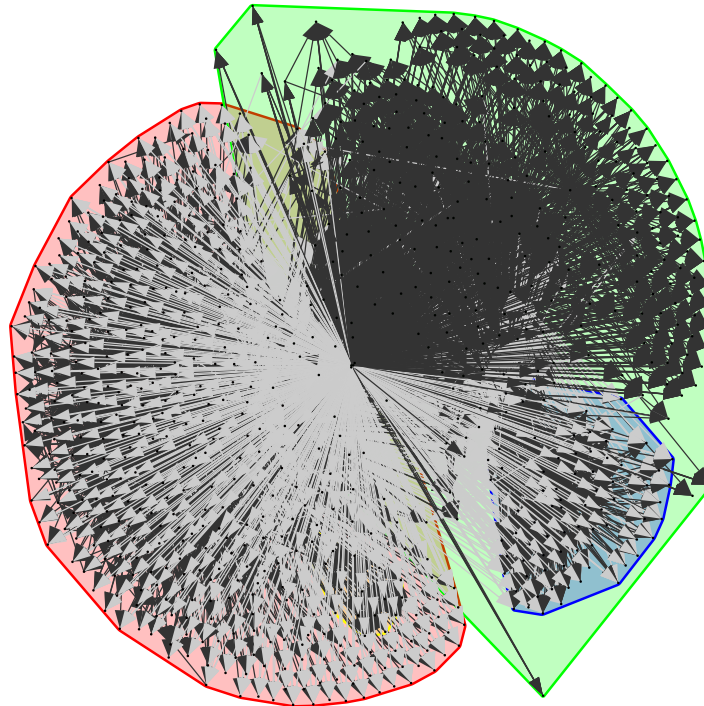
Answer: The in-degree and out-degree distributions of the 3 personal networks share similar trend, where lower degree has the higher frequency and as the degree increases, the frequency decreases.

2.1 1. Community structure of personal networks

2.1.1 Q20: For the 3 personal networks picked in Question 19, extract the community structure of each personal network using Walktrap community detection algorithm.

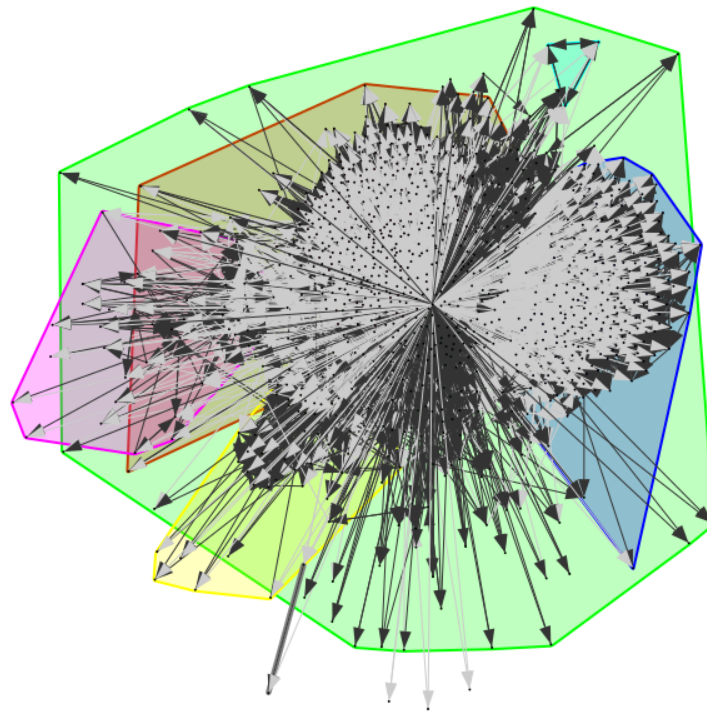
Report the modularity scores and plot the communities using colors. Are the modularity scores similar? In this question, you should have 3 plots.

Community plot for 109327480479767108490:



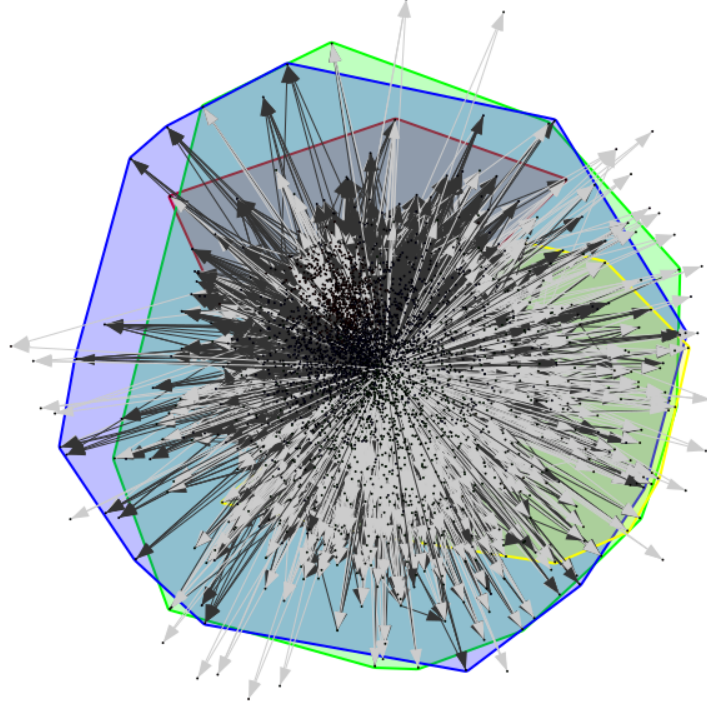
Modularity score for 109327480479767108490: 0.2752811260398488

Community plot for 115625564993990145546: 0.31996377165014245



Modularity score for 115625564993990145546: 0.31996377165014245

Community plot for 101373961279443806744: 0.1920449788191274



Modularity score for 101373961279443806744: 0.1920449788191274

Answer: The 3 community modularity scores are similar, showing some level of community structure. Here's summary of the results:

Node ID	Modularity
109327480479767108490	0.2753
115625564993990145546	0.3200
101373961279443806744	0.1920

2.1.2 Q21: Based on the expression for h and c, explain the meaning of homogeneity and completeness in words

- C is the set of circles, $C = \{C_1, C_2, C_3, \dots\}$
- K is the set of communities, $K = \{K_1, K_2, K_3, \dots\}$
- a_i is the number of people in circle C_i
- b_i is the number of people in community K_i with circle information
- N is the total number of people with circle information
- A_{ji} is the number of people belonging to community j and circle i

$$\text{Homogeneity} : h = 1 - \frac{H(C|K)}{H(C)}$$

$$\text{Completeness} : c = 1 - \frac{H(K|C)}{H(K)}$$

$$H(C) = - \sum_{i=1}^{|C|} \frac{a_i}{N} \log\left(\frac{a_i}{N}\right)$$

This $H(C)$ indicates measurement using the proportion of people in a circle. Circles with higher proportion is given a higher weight.

$$H(K) = - \sum_{i=1}^{|K|} \frac{b_i}{N} \log\left(\frac{b_i}{N}\right)$$

This $H(K)$ indicates measurement using the proportion of people in a community. communities with higher proportion is given a higher weight.

$$H(C|K) = - \sum_{j=1}^{|K|} \sum_{i=1}^{|C|} \frac{A_{ji}}{N} \log\left(\frac{A_{ji}}{b_j}\right)$$

This indicates the proportion of people in a community that are in a circle.

$$H(K|C) = - \sum_{i=1}^{|C|} \sum_{j=1}^{|K|} \frac{A_{ji}}{N} \log\left(\frac{A_{ji}}{a_i}\right)$$

This indicates the proportion of people in a circle that are in a community.

reference: <https://aclanthology.org/D07-1043.pdf>

Answer: Homogeneity and completeness are used to measure the quality of community structure by comparing circles and communities.

* Homogeneity (h) is 1 if each community contain only data points which are members of a single circles.

* Completeness (c) is 1 if all the data points that are members of a given circle are of the same community.

Increasing the homogeneity of a community often results in decreasing its completeness. This is because making communities more pure (with respect to circles) may require splitting circles across multiple communities, which reduces completeness.

2.1.3 Q22: Compute the h and c values for the community structures of the 3 personal network (same nodes as Question 19).

Interpret the values and provide a detailed explanation. Are there negative values? Why?

Answer: Summary of result:

Node Id	Circles	Communities	Homogeneity	Completeness	Modularity
109327480479767108490	3	4	0.7114	0.5947	0.2753
115625564993990145546	31	10	0.4223	0.5813	0.3200
101373961279443806744	3	31	0.0990	0.0398	0.1920

The network of 109327480479767108490 shows that it has the highest homogeneity (0.7114) and completeness (0.5947) among the 3. The plot from Q20 also confirm that there are overlapping communities. High homogeneity indicates that members of the same communities are from the same circle. Its completeness indicates that on average members of the same circles are splitted across different communities.

The network of 115625564993990145546 has ranked in the middle for homogeneity (0.4223) and completeness (0.5813) among the 3, meaning the members of the same community are mixed with the same circle and the members of the same circle are from different communities. The plot from Q20 indicates few large, dense communities. Modularity is the highest (0.3200) among the 3, suggesting a stronger community structure.

The network of 101373961279443806744 has the lowest modularity (0.1920), homogeneity (0.0990) and completeness (0.0398) among the 3. It suggests a weak relationship between communities and circles. It is nearly random and possible indication of heavy noise. The plot from Q20 shows that many data points are not part of any community.

None of the three networks produced negative values. Since homogeneity and completeness are based on entropy calculations and are defined as ratios bounded between 0 and 1, their values cannot be negative under the given formulas.

3 3. Cora dataset

3.0.1 Q23: Idea 1 - GCN

Node Classification problem - predict label of node

X shape: (2708, 1433)

Number of nodes (N): 2708

Number of features (F) of each node: 1433

Categories: {'Case_Based', 'Reinforcement_Learning', 'Probabilistic_Methods', 'Neural_Networks', 'Theory', 'Rule_Learning', 'Genetic_Algorithms'}

Number of classes: 7

Graph info: Graph with 2708 nodes and 5278 edges

Average Degree: 3.8980797636632203

GCC size: 2485

Case_Based 298

Reinforcement_Learning 217

Probabilistic_Methods 426

Neural_Networks 818

Theory 351

Rule_Learning 180
 Genetic_Algorithms 418
 Train set size: (140, 1433) (140, 7)
 Test set size: (2568, 1433) (2568, 7)

Layer (type)	Output Shape	Param #	Connected to
input_layer_4 (InputLayer)	(None, 1433)	0	-
input_layer_5 (InputLayer)	(None, None)	0	-
gcn_conv_custom_4 (GCNConvCustom)	(None, 16)	22,928	input_layer_4[0]... input_layer_5[0]...
dropout_2 (Dropout)	(None, 16)	0	gcn_conv_custom_...
gcn_conv_custom_5 (GCNConvCustom)	(None, 7)	112	dropout_2[0][0], input_layer_5[0]...

Total params: 23,040 (90.00 KB)

Trainable params: 23,040 (90.00 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/200

1/1 1s 886ms/step -
 accuracy: 0.1566 - loss: 0.1884 - val_accuracy: 0.2984 - val_loss: 0.2410

Epoch 2/200

1/1 0s 176ms/step -
 accuracy: 0.2511 - loss: 0.1602 - val_accuracy: 0.5140 - val_loss: 0.1490

Epoch 3/200

1/1 0s 164ms/step -
 accuracy: 0.4775 - loss: 0.0965 - val_accuracy: 0.5739 - val_loss: 0.1481

Epoch 4/200

1/1 0s 171ms/step -
 accuracy: 0.4889 - loss: 0.0797 - val_accuracy: 0.5916 - val_loss: 0.1432

Epoch 5/200

1/1 0s 194ms/step -
 accuracy: 0.5133 - loss: 0.0801 - val_accuracy: 0.6156 - val_loss: 0.1441

Epoch 6/200

1/1 0s 153ms/step -

accuracy: 0.4874 - loss: 0.0877 - val_accuracy: 0.6617 - val_loss: 0.1332
Epoch 7/200
1/1 0s 151ms/step -
accuracy: 0.5343 - loss: 0.0718 - val_accuracy: 0.6514 - val_loss: 0.1345
Epoch 8/200
1/1 0s 151ms/step -
accuracy: 0.5432 - loss: 0.0681 - val_accuracy: 0.6654 - val_loss: 0.1291
Epoch 9/200
1/1 0s 156ms/step -
accuracy: 0.5495 - loss: 0.0659 - val_accuracy: 0.6891 - val_loss: 0.1242
Epoch 10/200
1/1 0s 172ms/step -
accuracy: 0.5727 - loss: 0.0648 - val_accuracy: 0.6983 - val_loss: 0.1225
Epoch 11/200
1/1 0s 144ms/step -
accuracy: 0.5816 - loss: 0.0593 - val_accuracy: 0.7053 - val_loss: 0.1216
Epoch 12/200
1/1 0s 164ms/step -
accuracy: 0.6082 - loss: 0.0646 - val_accuracy: 0.7182 - val_loss: 0.1211
Epoch 13/200
1/1 0s 141ms/step -
accuracy: 0.5923 - loss: 0.0655 - val_accuracy: 0.7282 - val_loss: 0.1207
Epoch 14/200
1/1 0s 147ms/step -
accuracy: 0.5986 - loss: 0.0617 - val_accuracy: 0.7352 - val_loss: 0.1208
Epoch 15/200
1/1 0s 146ms/step -
accuracy: 0.6241 - loss: 0.0572 - val_accuracy: 0.7349 - val_loss: 0.1202
Epoch 16/200
1/1 0s 153ms/step -
accuracy: 0.6259 - loss: 0.0575 - val_accuracy: 0.7326 - val_loss: 0.1206
Epoch 17/200
1/1 0s 159ms/step -
accuracy: 0.6296 - loss: 0.0587 - val_accuracy: 0.7378 - val_loss: 0.1206
Epoch 18/200
1/1 0s 145ms/step -
accuracy: 0.6366 - loss: 0.0572 - val_accuracy: 0.7360 - val_loss: 0.1213
Epoch 19/200
1/1 0s 151ms/step -
accuracy: 0.6392 - loss: 0.0537 - val_accuracy: 0.7478 - val_loss: 0.1209
Epoch 20/200
1/1 0s 163ms/step -
accuracy: 0.6411 - loss: 0.0558 - val_accuracy: 0.7507 - val_loss: 0.1211
Epoch 21/200
1/1 0s 200ms/step -
accuracy: 0.6436 - loss: 0.0555 - val_accuracy: 0.7367 - val_loss: 0.1214
Epoch 22/200
1/1 0s 166ms/step -

accuracy: 0.6219 - loss: 0.0496 - val_accuracy: 0.7349 - val_loss: 0.1215
 Epoch 23/200
 1/1 0s 156ms/step -
 accuracy: 0.6208 - loss: 0.0516 - val_accuracy: 0.7378 - val_loss: 0.1217
 Epoch 24/200
 1/1 0s 156ms/step -
 accuracy: 0.6171 - loss: 0.0526 - val_accuracy: 0.7533 - val_loss: 0.1204
 Epoch 25/200
 1/1 0s 147ms/step -
 accuracy: 0.6340 - loss: 0.0485 - val_accuracy: 0.7566 - val_loss: 0.1184
 Epoch 26/200
 1/1 0s 142ms/step -
 accuracy: 0.6436 - loss: 0.0509 - val_accuracy: 0.7530 - val_loss: 0.1176
 Epoch 27/200
 1/1 0s 145ms/step -
 accuracy: 0.6462 - loss: 0.0501 - val_accuracy: 0.7134 - val_loss: 0.1294
 Epoch 28/200
 1/1 0s 181ms/step -
 accuracy: 0.6296 - loss: 0.0462 - val_accuracy: 0.6699 - val_loss: 0.1727
 Epoch 29/200
 1/1 0s 229ms/step -
 accuracy: 0.5883 - loss: 0.0818 - val_accuracy: 0.7338 - val_loss: 0.1252
 Epoch 30/200
 1/1 0s 153ms/step -
 accuracy: 0.6640 - loss: 0.0468 - val_accuracy: 0.7629 - val_loss: 0.1144
 Epoch 31/200
 1/1 0s 147ms/step -
 accuracy: 0.6603 - loss: 0.0472 - val_accuracy: 0.7710 - val_loss: 0.1134
 Epoch 32/200
 1/1 0s 145ms/step -
 accuracy: 0.6747 - loss: 0.0494 - val_accuracy: 0.7718 - val_loss: 0.1135
 Epoch 33/200
 1/1 0s 150ms/step -
 accuracy: 0.6942 - loss: 0.0489 - val_accuracy: 0.7699 - val_loss: 0.1144
 Epoch 34/200
 1/1 0s 146ms/step -
 accuracy: 0.6902 - loss: 0.0439 - val_accuracy: 0.7614 - val_loss: 0.1166
 Epoch 35/200
 1/1 0s 154ms/step -
 accuracy: 0.6843 - loss: 0.0472 - val_accuracy: 0.7559 - val_loss: 0.1195
 Epoch 36/200
 1/1 0s 180ms/step -
 accuracy: 0.6647 - loss: 0.0459 - val_accuracy: 0.7511 - val_loss: 0.1185
 Epoch 37/200
 1/1 0s 165ms/step -
 accuracy: 0.6569 - loss: 0.0455 - val_accuracy: 0.7522 - val_loss: 0.1126
 Epoch 38/200
 1/1 0s 147ms/step -

accuracy: 0.6743 - loss: 0.0442 - val_accuracy: 0.7504 - val_loss: 0.1113
Epoch 39/200
1/1 0s 151ms/step -
accuracy: 0.6532 - loss: 0.0441 - val_accuracy: 0.7408 - val_loss: 0.1146
Epoch 40/200
1/1 0s 152ms/step -
accuracy: 0.6496 - loss: 0.0419 - val_accuracy: 0.7467 - val_loss: 0.1140
Epoch 41/200
1/1 0s 161ms/step -
accuracy: 0.6640 - loss: 0.0407 - val_accuracy: 0.7493 - val_loss: 0.1135
Epoch 42/200
1/1 0s 143ms/step -
accuracy: 0.6222 - loss: 0.0417 - val_accuracy: 0.7507 - val_loss: 0.1131
Epoch 43/200
1/1 0s 157ms/step -
accuracy: 0.6573 - loss: 0.0404 - val_accuracy: 0.7581 - val_loss: 0.1135
Epoch 44/200
1/1 0s 152ms/step -
accuracy: 0.6326 - loss: 0.0374 - val_accuracy: 0.7559 - val_loss: 0.1142
Epoch 45/200
1/1 0s 141ms/step -
accuracy: 0.6226 - loss: 0.0393 - val_accuracy: 0.7489 - val_loss: 0.1159
Epoch 46/200
1/1 0s 143ms/step -
accuracy: 0.6518 - loss: 0.0427 - val_accuracy: 0.7308 - val_loss: 0.1186
Epoch 47/200
1/1 0s 153ms/step -
accuracy: 0.6267 - loss: 0.0411 - val_accuracy: 0.7278 - val_loss: 0.1199
Epoch 48/200
1/1 0s 158ms/step -
accuracy: 0.6171 - loss: 0.0496 - val_accuracy: 0.7489 - val_loss: 0.1139
Epoch 49/200
1/1 0s 150ms/step -
accuracy: 0.6470 - loss: 0.0428 - val_accuracy: 0.7400 - val_loss: 0.1127
Epoch 50/200
1/1 0s 163ms/step -
accuracy: 0.6606 - loss: 0.0390 - val_accuracy: 0.7301 - val_loss: 0.1140
Epoch 51/200
1/1 0s 146ms/step -
accuracy: 0.6636 - loss: 0.0371 - val_accuracy: 0.7175 - val_loss: 0.1185
Epoch 52/200
1/1 0s 146ms/step -
accuracy: 0.6363 - loss: 0.0342 - val_accuracy: 0.7275 - val_loss: 0.1126
Epoch 53/200
1/1 0s 165ms/step -
accuracy: 0.6418 - loss: 0.0368 - val_accuracy: 0.7330 - val_loss: 0.1130
Epoch 54/200
1/1 0s 154ms/step -

accuracy: 0.6462 - loss: 0.0373 - val_accuracy: 0.7341 - val_loss: 0.1120
Epoch 55/200
1/1 0s 183ms/step -
accuracy: 0.6695 - loss: 0.0375 - val_accuracy: 0.7363 - val_loss: 0.1101
Epoch 56/200
1/1 0s 151ms/step -
accuracy: 0.6451 - loss: 0.0355 - val_accuracy: 0.7422 - val_loss: 0.1080
Epoch 57/200
1/1 0s 157ms/step -
accuracy: 0.6525 - loss: 0.0376 - val_accuracy: 0.7541 - val_loss: 0.1051
Epoch 58/200
1/1 0s 185ms/step -
accuracy: 0.6758 - loss: 0.0341 - val_accuracy: 0.7633 - val_loss: 0.1029
Epoch 59/200
1/1 0s 148ms/step -
accuracy: 0.6614 - loss: 0.0330 - val_accuracy: 0.7640 - val_loss: 0.1023
Epoch 60/200
1/1 0s 213ms/step -
accuracy: 0.6806 - loss: 0.0333 - val_accuracy: 0.7688 - val_loss: 0.1016
Epoch 61/200
1/1 0s 225ms/step -
accuracy: 0.6924 - loss: 0.0326 - val_accuracy: 0.7659 - val_loss: 0.1030
Epoch 62/200
1/1 0s 187ms/step -
accuracy: 0.6854 - loss: 0.0354 - val_accuracy: 0.7496 - val_loss: 0.1057
Epoch 63/200
1/1 0s 154ms/step -
accuracy: 0.6743 - loss: 0.0328 - val_accuracy: 0.7430 - val_loss: 0.1068
Epoch 64/200
1/1 0s 146ms/step -
accuracy: 0.6695 - loss: 0.0325 - val_accuracy: 0.7448 - val_loss: 0.1073
Epoch 65/200
1/1 1s 606ms/step -
accuracy: 0.6385 - loss: 0.0317 - val_accuracy: 0.7448 - val_loss: 0.1086
Epoch 66/200
1/1 1s 1s/step -
accuracy: 0.6451 - loss: 0.0302 - val_accuracy: 0.7415 - val_loss: 0.1101
Epoch 67/200
1/1 2s 2s/step -
accuracy: 0.6569 - loss: 0.0310 - val_accuracy: 0.7467 - val_loss: 0.1118
Epoch 68/200
1/1 2s 2s/step -
accuracy: 0.6488 - loss: 0.0309 - val_accuracy: 0.7493 - val_loss: 0.1129
Epoch 69/200
1/1 3s 3s/step -
accuracy: 0.6499 - loss: 0.0296 - val_accuracy: 0.7578 - val_loss: 0.1129
Epoch 70/200
1/1 1s 1s/step -

accuracy: 0.6581 - loss: 0.0295 - val_accuracy: 0.7611 - val_loss: 0.1126
Epoch 71/200
1/1 1s 1s/step -
accuracy: 0.6532 - loss: 0.0313 - val_accuracy: 0.7659 - val_loss: 0.1102
Epoch 72/200
1/1 1s 987ms/step -
accuracy: 0.6673 - loss: 0.0280 - val_accuracy: 0.7696 - val_loss: 0.1090
Epoch 73/200
1/1 0s 154ms/step -
accuracy: 0.6813 - loss: 0.0295 - val_accuracy: 0.7722 - val_loss: 0.1076
Epoch 74/200
1/1 0s 150ms/step -
accuracy: 0.6710 - loss: 0.0289 - val_accuracy: 0.7736 - val_loss: 0.1075
Epoch 75/200
1/1 0s 146ms/step -
accuracy: 0.6677 - loss: 0.0299 - val_accuracy: 0.7781 - val_loss: 0.1078
Epoch 76/200
1/1 0s 148ms/step -
accuracy: 0.6887 - loss: 0.0283 - val_accuracy: 0.7806 - val_loss: 0.1070
Epoch 77/200
1/1 0s 157ms/step -
accuracy: 0.7027 - loss: 0.0266 - val_accuracy: 0.7806 - val_loss: 0.1070
Epoch 78/200
1/1 0s 159ms/step -
accuracy: 0.6798 - loss: 0.0265 - val_accuracy: 0.7799 - val_loss: 0.1077
Epoch 79/200
1/1 0s 165ms/step -
accuracy: 0.7016 - loss: 0.0276 - val_accuracy: 0.7799 - val_loss: 0.1083
Epoch 80/200
1/1 0s 152ms/step -
accuracy: 0.6998 - loss: 0.0287 - val_accuracy: 0.7799 - val_loss: 0.1083
Epoch 81/200
1/1 0s 150ms/step -
accuracy: 0.7083 - loss: 0.0252 - val_accuracy: 0.7762 - val_loss: 0.1097
Epoch 82/200
1/1 0s 149ms/step -
accuracy: 0.6953 - loss: 0.0274 - val_accuracy: 0.7674 - val_loss: 0.1115
Epoch 83/200
1/1 0s 162ms/step -
accuracy: 0.6950 - loss: 0.0242 - val_accuracy: 0.7589 - val_loss: 0.1144
Epoch 84/200
1/1 0s 150ms/step -
accuracy: 0.6747 - loss: 0.0291 - val_accuracy: 0.7629 - val_loss: 0.1138
Epoch 85/200
1/1 0s 161ms/step -
accuracy: 0.6942 - loss: 0.0245 - val_accuracy: 0.7677 - val_loss: 0.1133
Epoch 86/200
1/1 0s 178ms/step -

accuracy: 0.7097 - loss: 0.0258 - val_accuracy: 0.7740 - val_loss: 0.1122
Epoch 87/200
1/1 0s 141ms/step -
accuracy: 0.7016 - loss: 0.0277 - val_accuracy: 0.7814 - val_loss: 0.1117
Epoch 88/200
1/1 0s 145ms/step -
accuracy: 0.6809 - loss: 0.0273 - val_accuracy: 0.7840 - val_loss: 0.1120
Epoch 89/200
1/1 0s 144ms/step -
accuracy: 0.6902 - loss: 0.0273 - val_accuracy: 0.7855 - val_loss: 0.1119
Epoch 90/200
1/1 0s 143ms/step -
accuracy: 0.7064 - loss: 0.0278 - val_accuracy: 0.7806 - val_loss: 0.1126
Epoch 91/200
1/1 0s 165ms/step -
accuracy: 0.6942 - loss: 0.0238 - val_accuracy: 0.7777 - val_loss: 0.1135
Epoch 92/200
1/1 0s 152ms/step -
accuracy: 0.6846 - loss: 0.0302 - val_accuracy: 0.7747 - val_loss: 0.1085
Epoch 93/200
1/1 0s 159ms/step -
accuracy: 0.6976 - loss: 0.0231 - val_accuracy: 0.7799 - val_loss: 0.1046
Epoch 94/200
1/1 0s 164ms/step -
accuracy: 0.7035 - loss: 0.0239 - val_accuracy: 0.7806 - val_loss: 0.1024
Epoch 95/200
1/1 0s 150ms/step -
accuracy: 0.6784 - loss: 0.0249 - val_accuracy: 0.7777 - val_loss: 0.1015
Epoch 96/200
1/1 0s 148ms/step -
accuracy: 0.7038 - loss: 0.0261 - val_accuracy: 0.7688 - val_loss: 0.1009
Epoch 97/200
1/1 0s 152ms/step -
accuracy: 0.6824 - loss: 0.0269 - val_accuracy: 0.7729 - val_loss: 0.0999
Epoch 98/200
1/1 0s 180ms/step -
accuracy: 0.6821 - loss: 0.0292 - val_accuracy: 0.7707 - val_loss: 0.1006
Epoch 99/200
1/1 0s 152ms/step -
accuracy: 0.6725 - loss: 0.0235 - val_accuracy: 0.7747 - val_loss: 0.1012
Epoch 100/200
1/1 0s 156ms/step -
accuracy: 0.7031 - loss: 0.0229 - val_accuracy: 0.7644 - val_loss: 0.1037
Epoch 101/200
1/1 0s 149ms/step -
accuracy: 0.7005 - loss: 0.0246 - val_accuracy: 0.7614 - val_loss: 0.1065
Epoch 102/200
1/1 0s 168ms/step -

accuracy: 0.6968 - loss: 0.0264 - val_accuracy: 0.7618 - val_loss: 0.1100
Epoch 103/200
1/1 0s 150ms/step -
accuracy: 0.6673 - loss: 0.0272 - val_accuracy: 0.7633 - val_loss: 0.1107
Epoch 104/200
1/1 0s 155ms/step -
accuracy: 0.6824 - loss: 0.0229 - val_accuracy: 0.7659 - val_loss: 0.1111
Epoch 105/200
1/1 0s 149ms/step -
accuracy: 0.7020 - loss: 0.0221 - val_accuracy: 0.7688 - val_loss: 0.1124
Epoch 106/200
1/1 0s 155ms/step -
accuracy: 0.6998 - loss: 0.0245 - val_accuracy: 0.7659 - val_loss: 0.1136
Epoch 107/200
1/1 0s 153ms/step -
accuracy: 0.6990 - loss: 0.0250 - val_accuracy: 0.7655 - val_loss: 0.1136
Epoch 108/200
1/1 0s 151ms/step -
accuracy: 0.6569 - loss: 0.0228 - val_accuracy: 0.7688 - val_loss: 0.1128
Epoch 109/200
1/1 0s 152ms/step -
accuracy: 0.6876 - loss: 0.0233 - val_accuracy: 0.7729 - val_loss: 0.1108
Epoch 110/200
1/1 0s 155ms/step -
accuracy: 0.6972 - loss: 0.0211 - val_accuracy: 0.7747 - val_loss: 0.1096
Epoch 111/200
1/1 0s 173ms/step -
accuracy: 0.6872 - loss: 0.0227 - val_accuracy: 0.7544 - val_loss: 0.1132
Epoch 112/200
1/1 0s 149ms/step -
accuracy: 0.6802 - loss: 0.0230 - val_accuracy: 0.7858 - val_loss: 0.1085
Epoch 113/200
1/1 0s 153ms/step -
accuracy: 0.7297 - loss: 0.0210 - val_accuracy: 0.7862 - val_loss: 0.1077
Epoch 114/200
1/1 0s 155ms/step -
accuracy: 0.7031 - loss: 0.0239 - val_accuracy: 0.7799 - val_loss: 0.1120
Epoch 115/200
1/1 0s 167ms/step -
accuracy: 0.7049 - loss: 0.0231 - val_accuracy: 0.7666 - val_loss: 0.1161
Epoch 116/200
1/1 0s 155ms/step -
accuracy: 0.7001 - loss: 0.0287 - val_accuracy: 0.7655 - val_loss: 0.1104
Epoch 117/200
1/1 0s 149ms/step -
accuracy: 0.6983 - loss: 0.0289 - val_accuracy: 0.7710 - val_loss: 0.1037
Epoch 118/200
1/1 0s 146ms/step -

accuracy: 0.6994 - loss: 0.0216 - val_accuracy: 0.7755 - val_loss: 0.0998
Epoch 119/200
1/1 0s 145ms/step -
accuracy: 0.6987 - loss: 0.0207 - val_accuracy: 0.7688 - val_loss: 0.0992
Epoch 120/200
1/1 0s 143ms/step -
accuracy: 0.6857 - loss: 0.0229 - val_accuracy: 0.7633 - val_loss: 0.1003
Epoch 121/200
1/1 0s 152ms/step -
accuracy: 0.6629 - loss: 0.0251 - val_accuracy: 0.7544 - val_loss: 0.1027
Epoch 122/200
1/1 0s 144ms/step -
accuracy: 0.6651 - loss: 0.0230 - val_accuracy: 0.7496 - val_loss: 0.1043
Epoch 123/200
1/1 0s 165ms/step -
accuracy: 0.6987 - loss: 0.0261 - val_accuracy: 0.7478 - val_loss: 0.1058
Epoch 124/200
1/1 0s 152ms/step -
accuracy: 0.6773 - loss: 0.0259 - val_accuracy: 0.7470 - val_loss: 0.1071
Epoch 125/200
1/1 0s 174ms/step -
accuracy: 0.6654 - loss: 0.0237 - val_accuracy: 0.7470 - val_loss: 0.1094
Epoch 126/200
1/1 0s 150ms/step -
accuracy: 0.6839 - loss: 0.0251 - val_accuracy: 0.7489 - val_loss: 0.1102
Epoch 127/200
1/1 0s 160ms/step -
accuracy: 0.6606 - loss: 0.0234 - val_accuracy: 0.7563 - val_loss: 0.1097
Epoch 128/200
1/1 0s 158ms/step -
accuracy: 0.6872 - loss: 0.0247 - val_accuracy: 0.7622 - val_loss: 0.1106
Epoch 129/200
1/1 0s 185ms/step -
accuracy: 0.6817 - loss: 0.0264 - val_accuracy: 0.7522 - val_loss: 0.1157
Epoch 130/200
1/1 0s 154ms/step -
accuracy: 0.6942 - loss: 0.0244 - val_accuracy: 0.7352 - val_loss: 0.1222
Epoch 131/200
1/1 0s 151ms/step -
accuracy: 0.6625 - loss: 0.0234 - val_accuracy: 0.7367 - val_loss: 0.1242
Epoch 132/200
1/1 0s 184ms/step -
accuracy: 0.6865 - loss: 0.0240 - val_accuracy: 0.7437 - val_loss: 0.1244
Epoch 133/200
1/1 0s 145ms/step -
accuracy: 0.6732 - loss: 0.0232 - val_accuracy: 0.7474 - val_loss: 0.1234
Epoch 134/200
1/1 0s 152ms/step -

accuracy: 0.6632 - loss: 0.0240 - val_accuracy: 0.7614 - val_loss: 0.1192
Epoch 135/200
1/1 0s 157ms/step -
accuracy: 0.6725 - loss: 0.0222 - val_accuracy: 0.7589 - val_loss: 0.1175
Epoch 136/200
1/1 0s 144ms/step -
accuracy: 0.6942 - loss: 0.0212 - val_accuracy: 0.7648 - val_loss: 0.1163
Epoch 137/200
1/1 0s 146ms/step -
accuracy: 0.6761 - loss: 0.0239 - val_accuracy: 0.7622 - val_loss: 0.1193
Epoch 138/200
1/1 0s 150ms/step -
accuracy: 0.6843 - loss: 0.0272 - val_accuracy: 0.7651 - val_loss: 0.1179
Epoch 139/200
1/1 0s 146ms/step -
accuracy: 0.6813 - loss: 0.0240 - val_accuracy: 0.7592 - val_loss: 0.1181
Epoch 140/200
1/1 0s 171ms/step -
accuracy: 0.6917 - loss: 0.0230 - val_accuracy: 0.7526 - val_loss: 0.1186
Epoch 141/200
1/1 0s 180ms/step -
accuracy: 0.6976 - loss: 0.0210 - val_accuracy: 0.7552 - val_loss: 0.1181
Epoch 142/200
1/1 0s 155ms/step -
accuracy: 0.7038 - loss: 0.0225 - val_accuracy: 0.7559 - val_loss: 0.1173
Epoch 143/200
1/1 0s 156ms/step -
accuracy: 0.6780 - loss: 0.0236 - val_accuracy: 0.7555 - val_loss: 0.1178
Epoch 144/200
1/1 0s 157ms/step -
accuracy: 0.6839 - loss: 0.0245 - val_accuracy: 0.7581 - val_loss: 0.1166
Epoch 145/200
1/1 0s 142ms/step -
accuracy: 0.7094 - loss: 0.0231 - val_accuracy: 0.7581 - val_loss: 0.1154
Epoch 146/200
1/1 0s 162ms/step -
accuracy: 0.6584 - loss: 0.0266 - val_accuracy: 0.7415 - val_loss: 0.1233
Epoch 147/200
1/1 0s 168ms/step -
accuracy: 0.6776 - loss: 0.0216 - val_accuracy: 0.7219 - val_loss: 0.1275
Epoch 148/200
1/1 0s 168ms/step -
accuracy: 0.6584 - loss: 0.0210 - val_accuracy: 0.7216 - val_loss: 0.1284
Epoch 149/200
1/1 0s 152ms/step -
accuracy: 0.6544 - loss: 0.0236 - val_accuracy: 0.7253 - val_loss: 0.1268
Epoch 150/200
1/1 0s 151ms/step -

accuracy: 0.6588 - loss: 0.0217 - val_accuracy: 0.7312 - val_loss: 0.1243
Epoch 151/200
1/1 0s 149ms/step -
accuracy: 0.6355 - loss: 0.0256 - val_accuracy: 0.7386 - val_loss: 0.1236
Epoch 152/200
1/1 0s 146ms/step -
accuracy: 0.6569 - loss: 0.0214 - val_accuracy: 0.7445 - val_loss: 0.1226
Epoch 153/200
1/1 0s 164ms/step -
accuracy: 0.6758 - loss: 0.0227 - val_accuracy: 0.7474 - val_loss: 0.1228
Epoch 154/200
1/1 0s 153ms/step -
accuracy: 0.6647 - loss: 0.0250 - val_accuracy: 0.7489 - val_loss: 0.1248
Epoch 155/200
1/1 0s 155ms/step -
accuracy: 0.6595 - loss: 0.0222 - val_accuracy: 0.7493 - val_loss: 0.1236
Epoch 156/200
1/1 0s 171ms/step -
accuracy: 0.6821 - loss: 0.0229 - val_accuracy: 0.7515 - val_loss: 0.1236
Epoch 157/200
1/1 0s 147ms/step -
accuracy: 0.6750 - loss: 0.0200 - val_accuracy: 0.7507 - val_loss: 0.1235
Epoch 158/200
1/1 0s 147ms/step -
accuracy: 0.6636 - loss: 0.0226 - val_accuracy: 0.7448 - val_loss: 0.1247
Epoch 159/200
1/1 0s 147ms/step -
accuracy: 0.6665 - loss: 0.0210 - val_accuracy: 0.7437 - val_loss: 0.1252
Epoch 160/200
1/1 0s 150ms/step -
accuracy: 0.6721 - loss: 0.0230 - val_accuracy: 0.7655 - val_loss: 0.1150
Epoch 161/200
1/1 0s 167ms/step -
accuracy: 0.6957 - loss: 0.0205 - val_accuracy: 0.7404 - val_loss: 0.1216
Epoch 162/200
1/1 0s 169ms/step -
accuracy: 0.6577 - loss: 0.0214 - val_accuracy: 0.7286 - val_loss: 0.1251
Epoch 163/200
1/1 0s 165ms/step -
accuracy: 0.6747 - loss: 0.0241 - val_accuracy: 0.7219 - val_loss: 0.1275
Epoch 164/200
1/1 0s 146ms/step -
accuracy: 0.6507 - loss: 0.0256 - val_accuracy: 0.7142 - val_loss: 0.1240
Epoch 165/200
1/1 0s 160ms/step -
accuracy: 0.6551 - loss: 0.0177 - val_accuracy: 0.7190 - val_loss: 0.1212
Epoch 166/200
1/1 0s 153ms/step -

accuracy: 0.6573 - loss: 0.0224 - val_accuracy: 0.7260 - val_loss: 0.1167
Epoch 167/200
1/1 0s 151ms/step -
accuracy: 0.6632 - loss: 0.0225 - val_accuracy: 0.7312 - val_loss: 0.1135
Epoch 168/200
1/1 0s 154ms/step -
accuracy: 0.6621 - loss: 0.0271 - val_accuracy: 0.7334 - val_loss: 0.1120
Epoch 169/200
1/1 0s 182ms/step -
accuracy: 0.6610 - loss: 0.0235 - val_accuracy: 0.7345 - val_loss: 0.1118
Epoch 170/200
1/1 0s 143ms/step -
accuracy: 0.6880 - loss: 0.0233 - val_accuracy: 0.7371 - val_loss: 0.1122
Epoch 171/200
1/1 0s 150ms/step -
accuracy: 0.6710 - loss: 0.0228 - val_accuracy: 0.7459 - val_loss: 0.1124
Epoch 172/200
1/1 0s 156ms/step -
accuracy: 0.6909 - loss: 0.0213 - val_accuracy: 0.7537 - val_loss: 0.1128
Epoch 173/200
1/1 0s 162ms/step -
accuracy: 0.6928 - loss: 0.0213 - val_accuracy: 0.7655 - val_loss: 0.1099
Epoch 174/200
1/1 0s 199ms/step -
accuracy: 0.6990 - loss: 0.0248 - val_accuracy: 0.7729 - val_loss: 0.1080
Epoch 175/200
1/1 0s 172ms/step -
accuracy: 0.7116 - loss: 0.0203 - val_accuracy: 0.7777 - val_loss: 0.1074
Epoch 176/200
1/1 0s 159ms/step -
accuracy: 0.6953 - loss: 0.0219 - val_accuracy: 0.7829 - val_loss: 0.1071
Epoch 177/200
1/1 0s 167ms/step -
accuracy: 0.7061 - loss: 0.0211 - val_accuracy: 0.7825 - val_loss: 0.1047
Epoch 178/200
1/1 0s 152ms/step -
accuracy: 0.7020 - loss: 0.0237 - val_accuracy: 0.7792 - val_loss: 0.1051
Epoch 179/200
1/1 0s 151ms/step -
accuracy: 0.6990 - loss: 0.0204 - val_accuracy: 0.7758 - val_loss: 0.1080
Epoch 180/200
1/1 0s 149ms/step -
accuracy: 0.6902 - loss: 0.0231 - val_accuracy: 0.7910 - val_loss: 0.1038
Epoch 181/200
1/1 0s 164ms/step -
accuracy: 0.6990 - loss: 0.0232 - val_accuracy: 0.7925 - val_loss: 0.1034
Epoch 182/200
1/1 0s 150ms/step -

accuracy: 0.7031 - loss: 0.0190 - val_accuracy: 0.7866 - val_loss: 0.1054
Epoch 183/200
1/1 0s 151ms/step -
accuracy: 0.7083 - loss: 0.0224 - val_accuracy: 0.7818 - val_loss: 0.1084
Epoch 184/200
1/1 0s 142ms/step -
accuracy: 0.6869 - loss: 0.0230 - val_accuracy: 0.7766 - val_loss: 0.1126
Epoch 185/200
1/1 0s 155ms/step -
accuracy: 0.6640 - loss: 0.0332 - val_accuracy: 0.7581 - val_loss: 0.1146
Epoch 186/200
1/1 0s 151ms/step -
accuracy: 0.6883 - loss: 0.0205 - val_accuracy: 0.7463 - val_loss: 0.1195
Epoch 187/200
1/1 0s 163ms/step -
accuracy: 0.6592 - loss: 0.0227 - val_accuracy: 0.7563 - val_loss: 0.1184
Epoch 188/200
1/1 0s 162ms/step -
accuracy: 0.6599 - loss: 0.0218 - val_accuracy: 0.7563 - val_loss: 0.1184
Epoch 189/200
1/1 0s 185ms/step -
accuracy: 0.6665 - loss: 0.0232 - val_accuracy: 0.7522 - val_loss: 0.1205
Epoch 190/200
1/1 0s 155ms/step -
accuracy: 0.6643 - loss: 0.0232 - val_accuracy: 0.7404 - val_loss: 0.1247
Epoch 191/200
1/1 0s 163ms/step -
accuracy: 0.6632 - loss: 0.0235 - val_accuracy: 0.7356 - val_loss: 0.1286
Epoch 192/200
1/1 0s 160ms/step -
accuracy: 0.6713 - loss: 0.0240 - val_accuracy: 0.7374 - val_loss: 0.1305
Epoch 193/200
1/1 0s 175ms/step -
accuracy: 0.6798 - loss: 0.0228 - val_accuracy: 0.7408 - val_loss: 0.1310
Epoch 194/200
1/1 0s 157ms/step -
accuracy: 0.6787 - loss: 0.0226 - val_accuracy: 0.7493 - val_loss: 0.1292
Epoch 195/200
1/1 0s 158ms/step -
accuracy: 0.6769 - loss: 0.0221 - val_accuracy: 0.7544 - val_loss: 0.1287
Epoch 196/200
1/1 0s 163ms/step -
accuracy: 0.6614 - loss: 0.0241 - val_accuracy: 0.7640 - val_loss: 0.1226
Epoch 197/200
1/1 0s 165ms/step -
accuracy: 0.6725 - loss: 0.0243 - val_accuracy: 0.7703 - val_loss: 0.1179
Epoch 198/200
1/1 0s 153ms/step -

accuracy: 0.6857 - loss: 0.0263 - val_accuracy: 0.7692 - val_loss: 0.1154
Epoch 199/200

1/1 0s 155ms/step -

accuracy: 0.6990 - loss: 0.0261 - val_accuracy: 0.7670 - val_loss: 0.1141

Epoch 200/200

1/1 0s 152ms/step -

accuracy: 0.6965 - loss: 0.0223 - val_accuracy: 0.7677 - val_loss: 0.1132

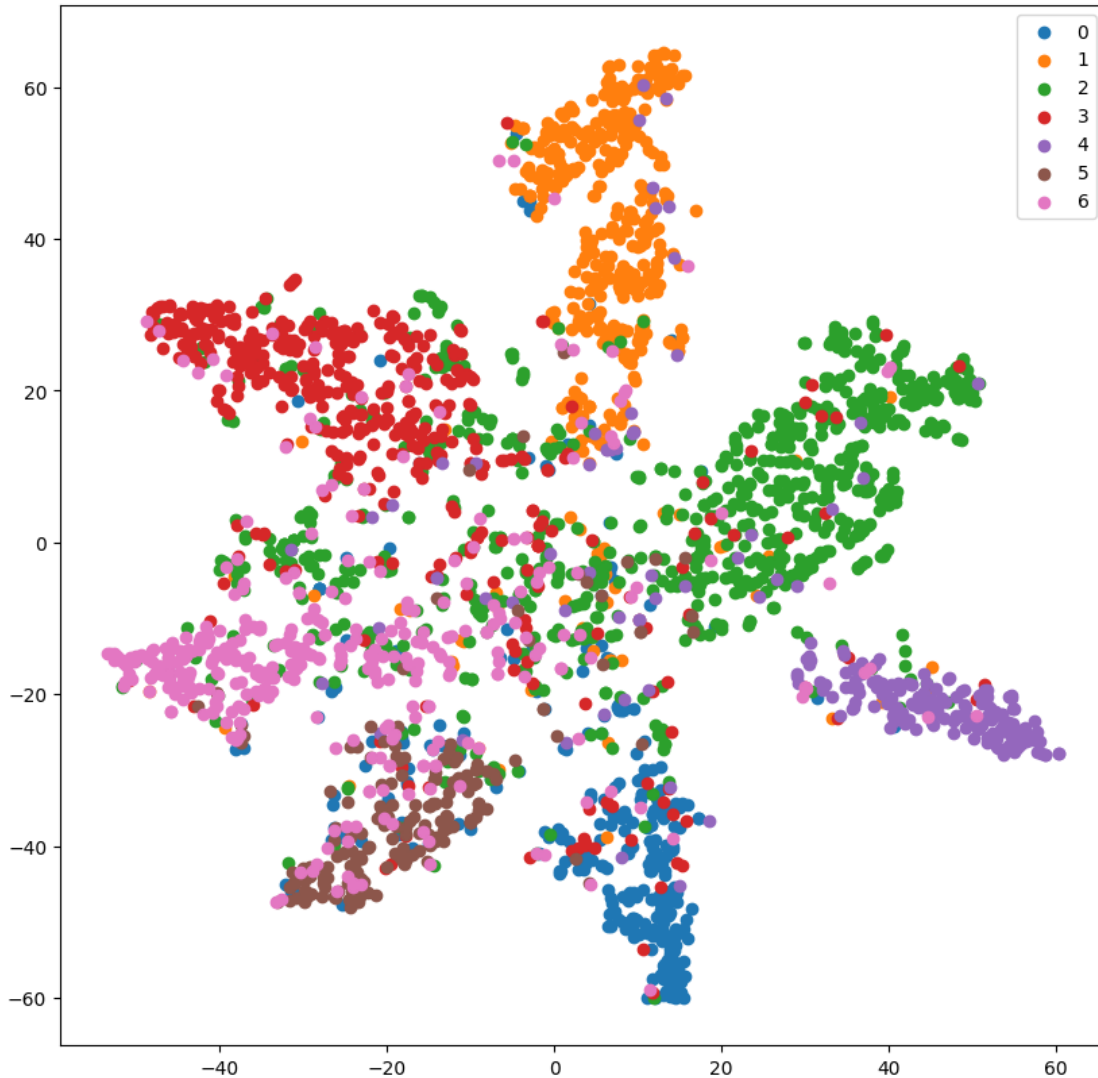
(2318, 1433) (2318, 2318) (2318, 7)

1/1 0s 74ms/step

(2318, 1433) (2318, 2318) (2318, 7) (2318, 7)

GCN Classification Report:

	precision	recall	f1-score	support
Case-Based	0.55	0.75	0.63	255
Reinforcement_Learning	0.90	0.69	0.78	358
Probabilistic_Methods	0.90	0.63	0.74	717
Neural_Networks	0.72	0.73	0.72	370
Theory	0.64	0.78	0.70	174
Rule_Learning	0.60	0.78	0.68	145
Genetic_Algorithms	0.51	0.71	0.59	299
accuracy			0.70	2318
macro avg	0.69	0.72	0.69	2318
weighted avg	0.74	0.70	0.71	2318



Q23: What hyperparameters do you choose to get the optimal performance? How many layers did you choose?

Answer: According to page 14 of the GCN reference paper, the accuracy plateaus between 2 and 7 layers before declining. Therefore, using 2 layers is sufficient. I experimented with 16 and 32 output channels for the first GCN layer using ReLU activation. The results show that 16 channels yield higher accuracy. Across different combinations of hyperparameter settings, the highest accuracy achieved was approximately 0.72.

For example, with 32, 16 channels:

channels = 32, accuracy: 0.6654 - loss: 0.0242 - val_accuracy: 0.7736 - val_loss: 0.1456

channels = 16, accuracy: 0.6965 - loss: 0.0223 - val_accuracy: 0.7677 - val_loss: 0.1132

3.0.2 Q24: Idea 2 - Node2Vec

StellarGraph: Undirected multigraph

Nodes: 2708, Edges: 5429

Node types:

paper: [2708]

Features: float32 vector, length 1433

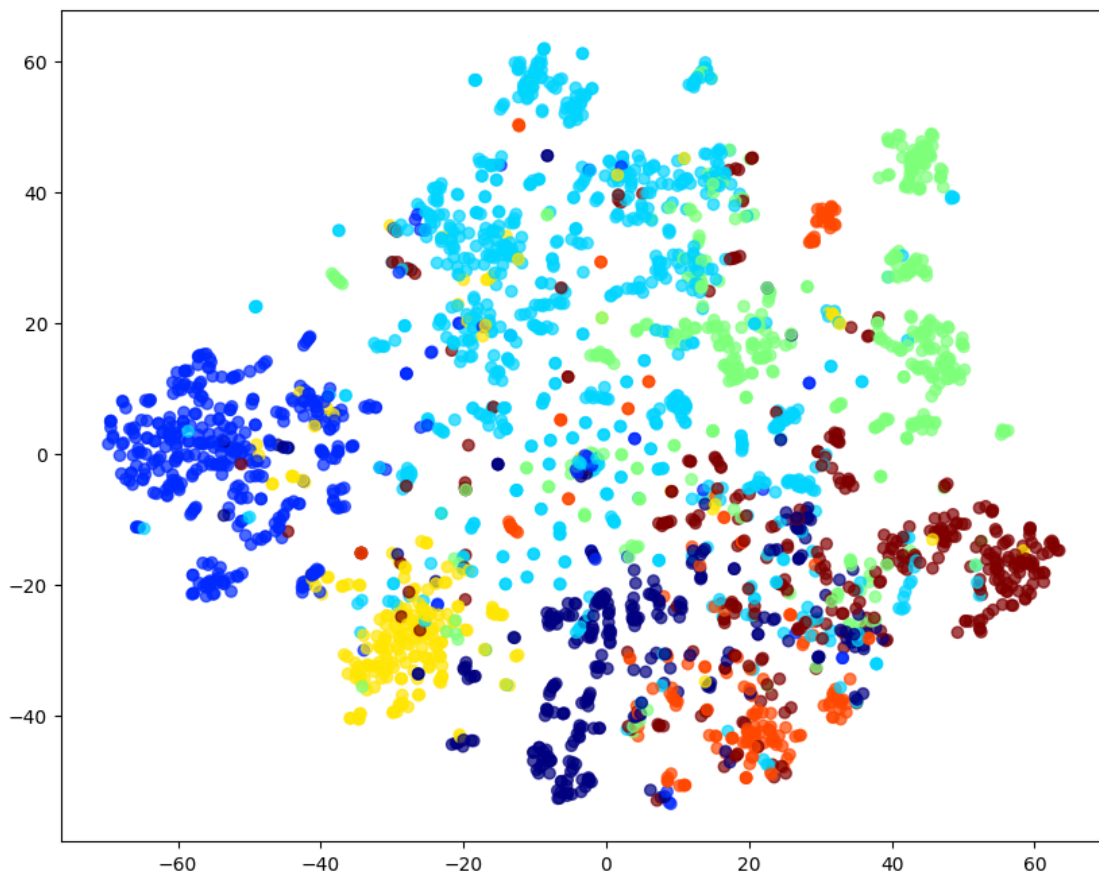
Edge types: paper-cites->paper

Edge types:

paper-cites->paper: [5429]

Weights: all 1 (default)

Features: none



Accuracy of Node2Vec features: 0.8154981549815498

Accuracy of Word_Vectors features: 0.7988929889298892

Accuracy of combined features: 0.8616236162361623

Answer: (Using Random Forest Classifier) As seen above, the accuracy when using the node2vec features is higher than the accuracy when using the word_vector features. This is likely due to the fact that the context within the node2vec features is retained and contributes to the categories we are classifying while the word_vector features only takes into account a limited number of words in a dictionary. Thus, the classifier with the node2vec features performs better than the word_vectors features.

The accuracy when using the combined node2vec and word_vectors features is much higher than the accuracy when using the node2vec or word_vector features alone. This is likely due to the fact that there is more data to classify the model on when combining the features than when using each set of features separately. Having more data to train on gives the model more to learn from, which will improve its ability to predict what the testing data should be classified as.

3.0.3 Q25: Idea 3 - PageRank

```
[13]: {'Case_Based': 0,
      'Genetic_Algorithms': 1,
      'Neural_Networks': 2,
      'Probabilistic_Methods': 3,
      'Reinforcement_Learning': 4,
      'Rule_Learning': 5,
      'Theory': 6}
```

X shape: (2708, 1433)

Number of nodes (N): 2708

Number of features (F) of each node: 1433

Categories: {0, 1, 2, 3, 4, 5, 6}

Number of classes: 7

Graph info: Graph with 2708 nodes and 5278 edges

```
[25]:   node  label  features
0      0      2  [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
1      1      5  [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, ...
2      2      4  [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
3      3      4  [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
4      4      3  [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
```

```
[31]:   node  label  features
0      0      2  [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
1      1      5  [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, ...
2      2      4  [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
3      3      4  [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
4      4      3  [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
```

```
[37]:
```

	node	label	features
172	167	0	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
818	770	0	[0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
799	752	0	[0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
47	47	0	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
1412	1332	0	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
...	...	...	...
555	523	6	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
1185	1117	6	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
1618	1525	6	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
518	488	6	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
1074	1013	6	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...

[140 rows x 3 columns]

PageRank - unvisited = 0

	precision	recall	f1-score	support
0	0.34	0.55	0.42	285
1	0.46	0.52	0.49	406
2	0.60	0.31	0.41	726
3	0.65	0.57	0.60	379
4	0.12	0.17	0.14	214
5	0.13	0.25	0.17	131
6	0.34	0.28	0.30	344
accuracy			0.39	2485
macro avg	0.38	0.38	0.36	2485
weighted avg	0.45	0.39	0.40	2485

- (a) teleportation takes the random walker to one of the seed documents of that class (with a uniform probability of 1/20 per seed document). Vary the teleportation probability in {0, 0.1, 0.2}.

For alpha = 0 - unvisited = 0

	precision	recall	f1-score	support
0	0.32	0.50	0.39	285
1	0.44	0.50	0.47	406
2	0.49	0.25	0.33	726
3	0.54	0.50	0.52	379
4	0.12	0.17	0.14	214
5	0.10	0.22	0.14	131
6	0.33	0.28	0.30	344
accuracy			0.35	2485
macro avg	0.34	0.35	0.33	2485
weighted avg	0.40	0.35	0.36	2485

For alpha = 0.1 - unvisited = 35

	precision	recall	f1-score	support
0	0.60	0.80	0.68	285
1	0.82	0.88	0.85	406
2	0.85	0.56	0.68	726
3	0.74	0.80	0.77	379
4	0.66	0.79	0.72	214
5	0.48	0.86	0.62	131
6	0.71	0.58	0.64	344
accuracy			0.72	2485
macro avg	0.69	0.76	0.71	2485
weighted avg	0.74	0.72	0.72	2485

For alpha = 0.2 - unvisited = 83

	precision	recall	f1-score	support
0	0.55	0.80	0.65	285
1	0.81	0.85	0.83	406
2	0.86	0.56	0.68	726
3	0.72	0.76	0.74	379
4	0.64	0.79	0.71	214
5	0.53	0.85	0.65	131
6	0.71	0.61	0.66	344
accuracy			0.71	2485
macro avg	0.69	0.75	0.70	2485
weighted avg	0.74	0.71	0.71	2485

- (b) the probability of transitioning to neighbors is not uniform among the neighbors. Rather, it is proportional to the cosine similarity between the text features of the current node and the next neighboring node.

$$p_i = \frac{\exp(x_0 x_i)}{\exp(x_0 x_1) + \exp(x_0 x_2) + \exp(x_0 x_3)}$$

; for i = 1, 2, 3.

FOR TFIDF - unvisited = 0

	precision	recall	f1-score	support
0	0.29	0.44	0.35	285
1	0.51	0.56	0.53	406
2	0.55	0.28	0.37	726
3	0.56	0.54	0.55	379
4	0.11	0.15	0.13	214

5	0.09	0.21	0.13	131
6	0.24	0.20	0.22	344
accuracy			0.36	2485
macro avg	0.34	0.34	0.33	2485
weighted avg	0.41	0.36	0.37	2485

For TFIDF with alpha = 0 , unvisited = 0

	precision	recall	f1-score	support
0	0.29	0.49	0.37	285
1	0.48	0.55	0.51	406
2	0.54	0.28	0.36	726
3	0.59	0.56	0.58	379
4	0.14	0.19	0.16	214
5	0.09	0.18	0.12	131
6	0.28	0.21	0.24	344
accuracy			0.37	2485
macro avg	0.34	0.35	0.33	2485
weighted avg	0.42	0.37	0.37	2485

For TFIDF with alpha = 0.1 , unvisited = 17

	precision	recall	f1-score	support
0	0.67	0.83	0.74	285
1	0.86	0.90	0.88	406
2	0.87	0.62	0.72	726
3	0.78	0.82	0.80	379
4	0.67	0.79	0.73	214
5	0.49	0.90	0.64	131
6	0.66	0.58	0.62	344
accuracy			0.74	2485
macro avg	0.71	0.78	0.73	2485
weighted avg	0.77	0.74	0.74	2485

For TFIDF with alpha = 0.2 , unvisited = 55

	precision	recall	f1-score	support
0	0.58	0.79	0.67	285
1	0.84	0.85	0.85	406
2	0.84	0.56	0.67	726
3	0.77	0.80	0.79	379
4	0.63	0.79	0.70	214
5	0.50	0.89	0.64	131
6	0.65	0.56	0.60	344

accuracy			0.71	2485
macro avg	0.69	0.75	0.70	2485
weighted avg	0.74	0.71	0.71	2485

3.0.4 Conclusion

Summary of Results:

Model	Teleportation	Unvisited	Accuracy	F1 Scores
PageRank	No tp	0	0.39	[.42, .49, .41, .60, .14, .17, .30]
PageRank	alpha=0	0	0.35	[.39, .47, .33, .52, .14, .14, .30]
PageRank	alpha=0.1	35	0.72	[.68, .85, .68, .77, .72, .62, .64]
PageRank	alpha=0.2	83	0.71	[.65, .83, .68, .74, .71, .65, .66]
TFIDF	No tp	0	0.39	[.35, .53, .37, .55, .13, .13, .22]
TFIDF	alpha=0	0	0.37	[.37, .51, .36, .58, .16, .12, .24]
TFIDF	alpha=0.1	17	0.74	[.74, .88, .72, .80, .73, .64, .62]
TFIDF	alpha=0.2	55	0.71	[.67, .85, .67, .79, .70, .64, .60]

Accuracy increases drastically when using teleportation with a small alpha in the page rank approach. Accuracy improved from <40% to 70%+ in this case. PageRank teleportation with a uniform probability versus using a weighted scale with cosine similarity of feature did not yield much improvement on accuracy. With TFIDF random walk, the number of unvisited node counts decreases. Also note the time it took to run this algorithm is significantly longer than the two previous approaches.

In conclusion, in addition to page rank accuracy above, training a 2-layer GCN resulted in a 70% accuracy, while using the Node2Vec approach achieve the highest accuracy of 86% in our node classification study.